

UNIVERZITET U BEOGRADU
ELEKTROTEHNIČKI FAKULTET



Efikasna hardverska realizacija IP jezgara za brzu Furijeovu transformaciju

Master rad

Mentor:
dr Vladimir Petrović, docent

Kandidat:
Radomir Vranjevac, 2021/3029

Beograd, Septembar 2023.

UNIVERSITY OF BELGRADE
SCHOOL OF ELECTRICAL ENGINEERING



Efficient FPGA implementation of Fast Fourier Transform IP cores

Master's Thesis

Advisor:
dr. Vladimir Petrović, assistant professor

Candidate:
Radomir Vranjevac, 2021/3029

Belgrade, September 2023

Contents

Contents	1
1 Introduction	3
2 Theoretical Background	5
2.1 Cooley-Tukey Algorithm	6
2.2 Inverse FFT	10
2.3 Radix-2 ² FFT	11
2.4 Fixed-Point Arithmetic	13
2.4.1 Rounding and Overflow	14
3 Software Reference Model	16
3.1 Architecture	16
3.1.1 Pre-adder	17
3.1.2 Rotator	18
4 Hardware Architecture	19
4.1 Pipelined Streaming FFT Architectures	19
4.1.1 SDF	20
4.1.2 SFF	22
4.1.3 MDF	23
4.2 Data Reordering Circuit	26
4.2.1 Serial Pipelined Architectures	26
4.2.2 Parallel Pipelined Architectures	27
5 Hardware Implementation	29
5.1 Stage-chain	29
5.2 Pre-adder	30
5.2.1 FIFO	30
5.2.2 Butterfly	31
5.3 Trivial Rotator	31
5.4 Complex Rotator	31
5.4.1 Twiddle ROM	32
5.4.2 Twiddle Address Generator	33
5.4.3 Twiddle Generator	33
5.4.4 Complex Multiplier	33
6 Results and Comparison	38
6.1 Hardware Utilization	38
6.2 SQNR Performance	39
6.3 Testing and Verification	41

7 Conclusion	44
Bibliography	45
List of Abbreviations	47
List of Figures	48
List of Tables	49

Chapter 1

Introduction

Ever since its invention, the FFT algorithm found use in thousands of applications, and is deemed one of the most important algorithms ever developed. This algorithm has a special significance in the fields of wireless and wired telecommunications, as a key component in modern telecommunication systems such as those based on orthogonal frequency-division multiplexing (OFDM) or single-carrier frequency-division multiple access (SC-FDMA). The push for ever higher data throughput of wireless telecommunication systems reflected on the design of the FFT IP cores, requiring high-frequency and high-parallelism implementations.

Traditionally, such high-throughput systems have been developed on application-specific integrated circuits (ASIC) which offer the best power usage and the highest performance. The problem with ASICs is the limited flexibility they provide, as the design cannot be changed after fabrication. Another big obstacle is the high cost of fabricating the circuits, which is typically done when a large number of devices is required. However, modern Field-Programmable Gate Array (FPGA) platforms provide devices which can implement a high-speed, high-throughput design, while having full flexibility and programmability. This has pushed many companies towards developing products on FPGA devices, such as the Zynq RFSoc line of devices from Xilinx.

To be able to implement an FFT on a such FPGA architecture efficiently, one must take many details of the platform into consideration. This is especially important for the efficient use of capabilities found in DSP units, as well as choosing the right type of memory as per the requirements.

The aim of this thesis is to present several different FFT architectures, noting their strengths and weaknesses. Based on the design requirements, a selection of architectures is made and those architectures are later modelled and implemented. Finally, the hardware platform is examined and a mapping of the architecture components to the available resources is presented.

The implementation is done in VHDL language using Xilinx Vivado 2021.1 design suite. The modelling is done using the Python programming language and represents a bit-exact fixed-point software model of the implementation.

In chapter 2, the theoretical background of the FFT algorithm is given. It is followed by a short description of a modification of the algorithm that decreases the hardware complexity. Finally, the fixed-point arithmetic is described, along with different options that have a great impact on the performance.

Chapter 3 first describes the library that was developed to implement fixed-point arithmetic, and later focuses on the details of the software model implementation.

A significant part in using the available resources efficiently is selecting the right FFT architecture. Therefore, chapter 4 focuses on describing different FFT architectures. Here a block design and hardware complexity of each of the architectures is analyzed. The final part of the chapter 4 discusses the circuits used for reordering of data samples at the output of the

FFTs from bit-reverse to the natural order.

Chapter 5 breaks down the architecture into smaller units that are easily implemented, and a mapping to the hardware resources is given.

Finally, chapter 6 shows the results of the implementation. In the first part, this refers to the hardware resources of the implemented design, comparing it to an equivalent IP core available from Xilinx. The second part of this chapter shows the SQNR performance of the implemented design for various configurations, which is most useful for determining the parameters that satisfy the requirements while maintaining the lowest possible hardware complexity. In the end, an overview of the setup used for verifying the design performance and a configuration used to test the design in hardware are shown.

Chapter 2

Theoretical Background

The Fourier Transform is a mathematical transform that is used to analyze the frequency content of a signal. It can be applied on a continuous signal, and is represented by the following formula:

$$F(\omega) = \int_{-\infty}^{\infty} f(t) e^{-j2\pi\omega t} dt \quad (2.1)$$

where $F(\omega)$ is a Fourier transform of the function $f(t)$. It is also said that the function $f(t)$ is in the time domain, while $F(\omega)$ is in the frequency domain.

Modern computer hardware is much more suited for processing discrete samples, rather than continuous signals which are harder to obtain and process. The Discrete Fourier Transform is a variation of the Fourier Transform that takes a discrete function as an input and produces a discrete function as an output. In the following formula $x[n]$ is the n -th input sample ($n \in \{0, \dots, N-1\}$); whereas $X[k]$ represents the k -th output sample ($k \in \{0, \dots, N-1\}$).

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi}{N}nk} \quad (2.2)$$

However, compared to the Fourier transform applied on a continuous signal, the Discrete Fourier transform has an unwanted effect known as aliasing. It manifests itself as an infinite number of copies of the signal in the spectral domain that are exactly $1/T_S$ apart, where T_S is the sampling period. This also imposes a limitation on minimum sampling frequency to avoid overlapping the spectral copies that are a consequence of sampling. This limitation is commonly known as the Nyquist criterion.

The factors $e^{-j\frac{2\pi}{N}nk}$ in the equation 2.2 are commonly called twiddle factors and are denoted by W_N^{nk} , such that:

$$W_N^{nk} = e^{-j\frac{2\pi}{N}nk} \quad (2.3)$$

The twiddle factors satisfy the identity shown in equation 2.4 using the definition of the twiddle factors.

$$W_N^{2nk} = e^{-j\frac{2\pi}{N}2nk} = e^{-j\frac{2\pi}{N/2}nk} = W_{N/2}^{nk} \quad (2.4)$$

The complexity of calculating DFT by definition is proportional to N^2 , where N is the number of samples. This is easily proven by counting the number of operations (additions and multiplications) that are required to calculate each output sample: $N-1$ complex additions and N complex multiplications are required, and there are N output samples that are to be calculated. The total number of complex additions and multiplications is given by the following equations:

$$\begin{aligned} R_A &= N \cdot (N - 1) \\ R_M &= N^2 \end{aligned} \tag{2.5}$$

The Fast Fourier Transform is an algorithm that computes the DFT, but does so with a lower computational complexity. There are numerous variations of how that is accomplished, but the most common one is known as the Cooley-Tukey algorithm, named after the authors [1]. Following is a brief description of the Cooley-Tukey algorithm.

2.1 Cooley-Tukey Algorithm

The Cooley-Tukey algorithm is derived by rearranging the calculation so that calculation sub-products are reused for other output samples, thus lowering the overall number of operations. There are two options for doing the rearrangement depending on whether the rearrangement is done at the input or at the output. Rearranging the input samples is an algorithm known as decimation in time (DIT), whereas rearranging the output samples is called decimation in frequency (DIF). Following is the derivation of the DIF algorithm.

The idea is to do a decomposition of N to two natural numbers N_1 and N_2 such that $N = N_1 \cdot N_2$, implementing DFT as a set of N_1 DFT units of size N_2 . A special case of that is for $N_1 = 2$, decomposing an N -point DFT into 2 DFTs of equal size. This is known as a radix-2 algorithm.

The following is true:

$$\begin{aligned} X[k] &= \sum_{n=0}^{N-1} x[n] \cdot W_N^{n \cdot k} \\ &= \sum_{n=0}^{N/2-1} x[n] \cdot W_N^{n \cdot k} + \sum_{n=N/2}^{N-1} x[n] \cdot W_N^{n \cdot k} \\ &= \sum_{n=0}^{N/2-1} x[n] \cdot W_N^{n \cdot k} + \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_N^{(n+N/2) \cdot k} \\ &= \sum_{n=0}^{N/2-1} x[n] \cdot W_N^{n \cdot k} + W_N^{N/2 \cdot k} \cdot \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_N^{n \cdot k} \end{aligned} \tag{2.6}$$

Calculating the last expression for $k = 2m$ and plugging in the equation 2.4, the even numbered output samples are calculated:

$$\begin{aligned} X[2m] &= \sum_{n=0}^{N/2-1} x[n] \cdot W_N^{n \cdot 2m} + W_N^{N/2 \cdot 2m} \cdot \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_N^{n \cdot 2m} \\ &= \sum_{n=0}^{N/2-1} x[n] \cdot W_{N/2}^{n \cdot m} + W_N^{N \cdot m} \cdot \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_{N/2}^{n \cdot m} \\ &= \sum_{n=0}^{N/2-1} (x[n] + x[n + N/2]) \cdot W_{N/2}^{n \cdot m} \end{aligned} \tag{2.7}$$

where $m \in \{0, \dots, N/2 - 1\}$.

If the function $U[m]$ is equal to $X[2m]$, then $U[m]$ can be viewed as a DFT of the signal $u[n]$ if $u[n]$ is defined in the following way:

$$u[n] = x[n] + x[n + N/2] \quad (2.8)$$

where $n \in \{0, \dots, N/2 - 1\}$. Calculating each sample of the function $u[n]$ requires a complex addition.

Similarly, the odd numbered samples are calculated by calculating the equation 2.6 when $k = 2m + 1$:

$$\begin{aligned} X[2m + 1] &= \sum_{n=0}^{N/2-1} x[n] \cdot W_N^{n \cdot (2m+1)} + W_N^{N/2 \cdot (2m+1)} \cdot \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_N^{n \cdot (2m+1)} \\ &= \sum_{n=0}^{N/2-1} x[n] \cdot W_N^{n \cdot 2m} \cdot W_N^n + W_N^{N/2 \cdot (2m+1)} \cdot \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_N^{n \cdot 2m} \cdot W_N^n \\ &= \sum_{n=0}^{N/2-1} x[n] \cdot W_{N/2}^{n \cdot m} \cdot W_N^n + W_N^{N/2 \cdot m} \cdot W_N^{N/2} \cdot \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_{N/2}^{n \cdot m} \cdot W_N^n \\ &= \sum_{n=0}^{N/2-1} \left((x[n] - x[n + N/2]) \cdot W_N^n \right) \cdot W_{N/2}^{n \cdot m} \end{aligned} \quad (2.9)$$

where $m \in \{0, \dots, N/2 - 1\}$.

The last equation shows that the function $X[2m + 1]$ can be viewed as the DFT of the signal $v[n]$ when $v[n]$ is defined by the following expression:

$$v[n] = (x[n] - x[n + N/2]) \cdot W_N^n \quad (2.10)$$

where $n \in \{0, \dots, N/2 - 1\}$. Calculating each sample of the function $v[n]$ requires a complex addition and a complex multiplication.

The given equations can be represented by the block diagram given in figure 2.1.

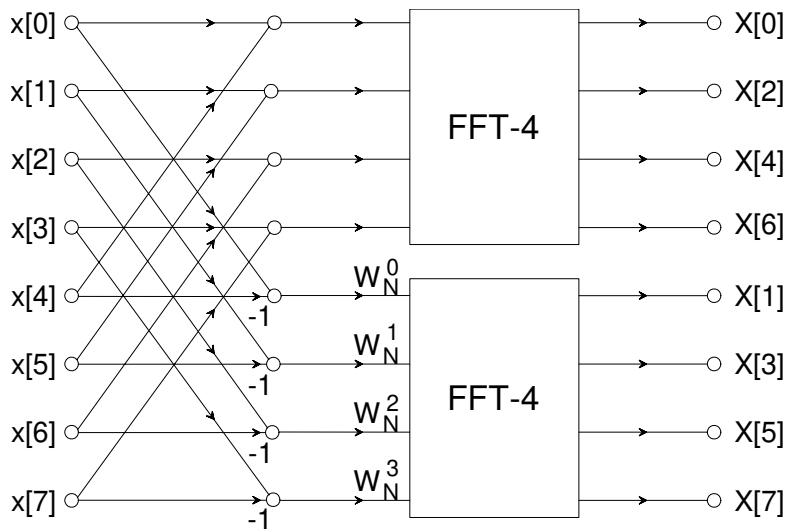


Figure 2.1: Signal flow graph of an FFT operation split into 2 smaller FFTs [2]

In figure 2.1 the numbers above the lines represent a complex multiplication with a corresponding twiddle factor, while the addition and subtraction are represented by a node that has two inputs and one output.

The operation of an addition and subtraction, where the subtraction is followed by a complex multiplication, is commonly known as a butterfly operation, and is shown in figure 2.2.

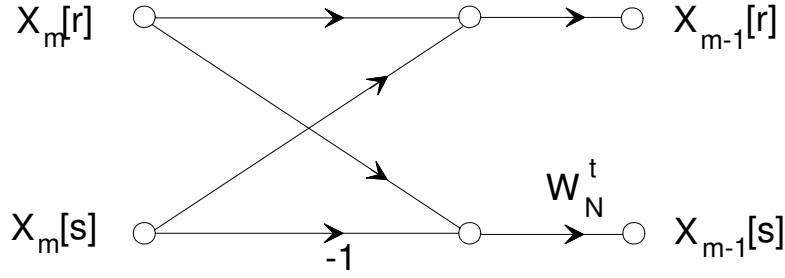


Figure 2.2: Butterfly of a radix-2 DIF FFT [2]

Equations 2.7 and 2.9 are an $N/2$ -point DFT of the functions $u[n]$ and $v[n]$, respectively. One $N/2$ -point DFT requires $N/2 \cdot (N/2 - 1)$ complex additions and $(N/2)^2$ complex multiplications to calculate, but operations for calculating $u[n]$ and $v[n]$ must be counted as well.

The total number of complex additions required to calculate the FFT is thus:

$$\begin{aligned} N_A &= 2 \cdot \frac{N}{2} (N/2 - 1) + N/2 + N/2 \\ &= N \cdot (N/2 - 1) + N \\ &= N^2/2 \end{aligned} \quad (2.11)$$

while the total number of complex multiplications is given by the following expression:

$$\begin{aligned} N_M &= 2 \cdot \frac{N^2}{2} + N/2 \\ &= \frac{N(N+1)}{2} \end{aligned} \quad (2.12)$$

Both the number of additions and the number of multiplications decreased when compared to the the number of operations required for calculating the DFT by definition (equation 2.5).

If N is a power of 2, the algorithm can be recursively applied until 2-point DFT is encountered. It can be proven that the number of operations of the resulting algorithm is proportional to $N \log N$ [1].

Such recursive decomposition yields the block diagram shown in figure 2.3.

This decomposes an FFT to a number of stages each consisting of butterfly units and complex multipliers. The number of stages depends on the FFT size N and is equal to $m = \log_2 N$. Stages are numbered starting at the input using a stage index s , where $s \in \{1, 2, \dots, m\}$.

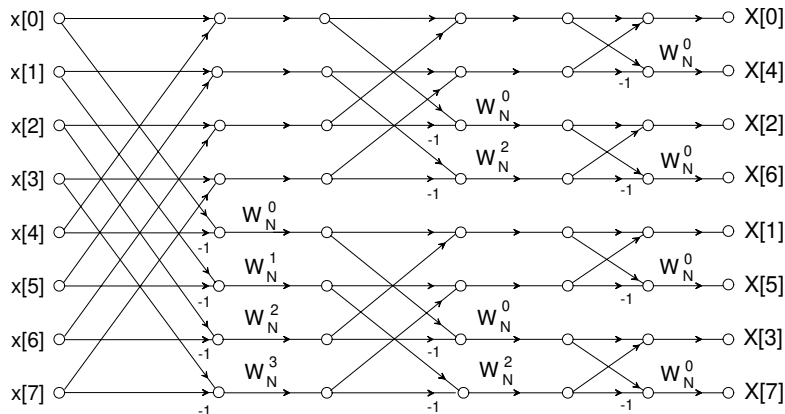


Figure 2.3: Signal flow graph of a DIF FFT [2]

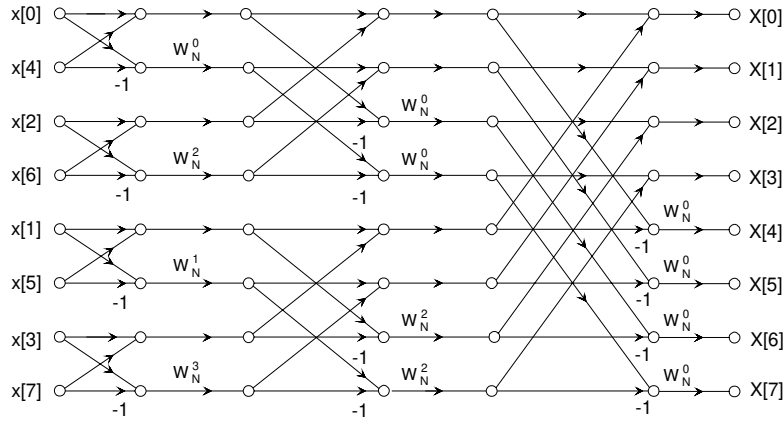


Figure 2.4: Signal flow graph of a DIF FFT with the input samples in bit-reverse order [2]

It can be noticed that applying the radix-2 DIF algorithm changes the order of the output samples. When the rearrangement is done, the output samples are sorted based on whether or not their index is even or odd, with even indices in the first half of the output samples and odd ones grouped in the second half. Looking at the binary representation of the indices, the least significant bit determines whether the sample ends up in the first or the second half of the output. Recursively applying the algorithm leads to reversing the order of bits in a binary representation of the indices. This is known as the bit-reverse order.

The signal flow graph of the DIF algorithm shown in figure 2.3 shows the operations that are done to each sample while calculating the output samples. It is possible to change the order of horizontal lines (while maintaining all the connections) to get the natural order of samples at the output. This in turn causes the order of input samples to become bit-reversed, and the resulting graph is shown in figure 2.4.

Data order reversal also caused the "size" of the butterfly units to change, as well as the order of the twiddle factors in each stage.

Thus, without data reordering operations, radix-2 DIF algorithm requires that either input or output order is bit-reversed. Data reordering is necessary if the same order at both input and output is required.

The DIT algorithm applies the same principles to rearrange the input samples, and results in the following equations:

$$X[k] = \sum_{m=0}^{N/2-1} \left(x[2m] + W_N^k \cdot x[2m+1] \right) \cdot W_{N/2}^{m \cdot k} \quad (2.13)$$

$$X[k + N/2] = \sum_{m=0}^{N/2-1} \left(x[2m] - W_N^k \cdot x[2m+1] \right) \cdot W_{N/2}^{m \cdot k} \quad (2.14)$$

where $k \in \{0, \dots, N/2 - 1\}$.

The DIT FFT algorithm can be represented by the signal flow graph shown in figure 2.5.

The resulting architecture is very similar to the architecture of the DIF algorithm, but the butterfly unit differs between the two architectures. The butterfly from a radix-2 DIT FFT is shown in figure 2.6. The butterfly unit of a DIT FFT is formed by multiplying one operand with a twiddle factor, and then doing an addition and a subtraction.

It can be shown that the total number of operations of a DIT FFT is exactly the same as for the FFT derived using the DIF algorithm [1].

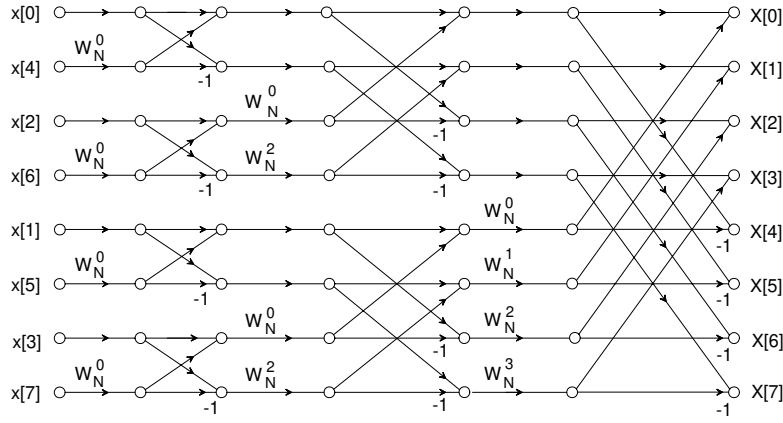


Figure 2.5: Signal flow graph of a DIT algorithm [2]

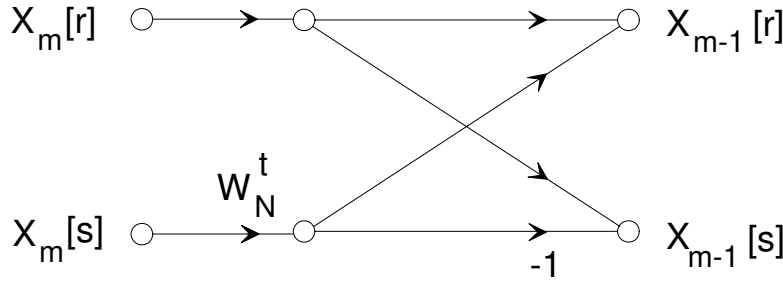


Figure 2.6: Butterfly of a radix-2 DIT FFT [2]

2.2 Inverse FFT

Only forward DFT has been discussed thus far, but an algorithm for transforming the data from the frequency domain back to time domain also exists. It is called inverse Fourier transform and it is defined using the following formula:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \cdot e^{j\frac{2\pi}{N}nk}, \quad n \in 0, \dots, N-1 \quad (2.15)$$

Aside from the scaling factor $\frac{1}{N}$, the only difference to the definition of the discrete Fourier transform is the complex conjugation of the twiddle factors. It can be shown through mathematical operations that the inverse Fourier transform can be calculated using the forward Fourier transform by conjugating the input to the FFT, and conjugating the output. This is shown in figure 2.7.

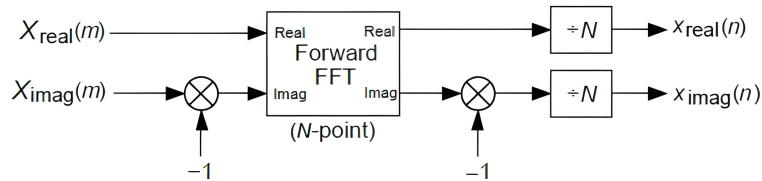


Figure 2.7: IFFT using the forward FFT by conjugating the data [3]

The drawback of the shown method is the calculation that has to be performed, possibly taking resources. The alternative is shown in figure 2.8, accomplishing the same goal by swapping the real and imaginary parts at the input and output of the FFT.

If a runtime configurable inverse/forward configuration is needed the first method is preferred as it takes less resources, whereas for a static configuration the second method takes no

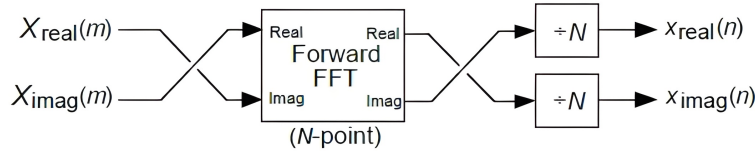


Figure 2.8: IFFT using the forward FFT by swapping the real and imaginary parts [3]

resources and is thus superior.

The scaling of data by $\frac{1}{N}$ is trivial when the FFT size is a power of 2, accomplished by moving the position of the binary point to the left by $\log_2 N$ bits.

2.3 Radix-2² FFT

The Cooley-Tukey algorithm reduced the overall number of operations by reusing some results for calculating several output samples. The algorithm can further be simplified if fusing two consecutive multiplications produces a complex multiplication that is trivial to implement. Trivial complex multiplications are the multiplications by 1, $1j$, -1 , and $-1j$. Multiplications by these factors can be implemented using simple operations such as sign inversion and/or interchanging the real and imaginary parts of the number.

So far, only radix-2 algorithms have been discussed. Other than that, higher radix algorithms such as radix-3 [4], radix-4 [5], prime-factor-radix [6], or even split-radix [7] and mixed-radix [8] algorithms exist. Compared to the radix-2 algorithm, higher radices such as radix-4 can achieve a lower multiplier complexity, while having a higher butterfly complexity [9].

Radix-2² is an algorithm that lowers the number of complex rotations in an FFT compared to the regular radix-2 algorithm. This is achieved by implementing a Common Factor Algorithm [10] instead of a regular radix-2 decomposition. The result is a signal flow graph shown in figure 2.9b, the only difference from figure 2.3 being the exact values of the twiddle factors.

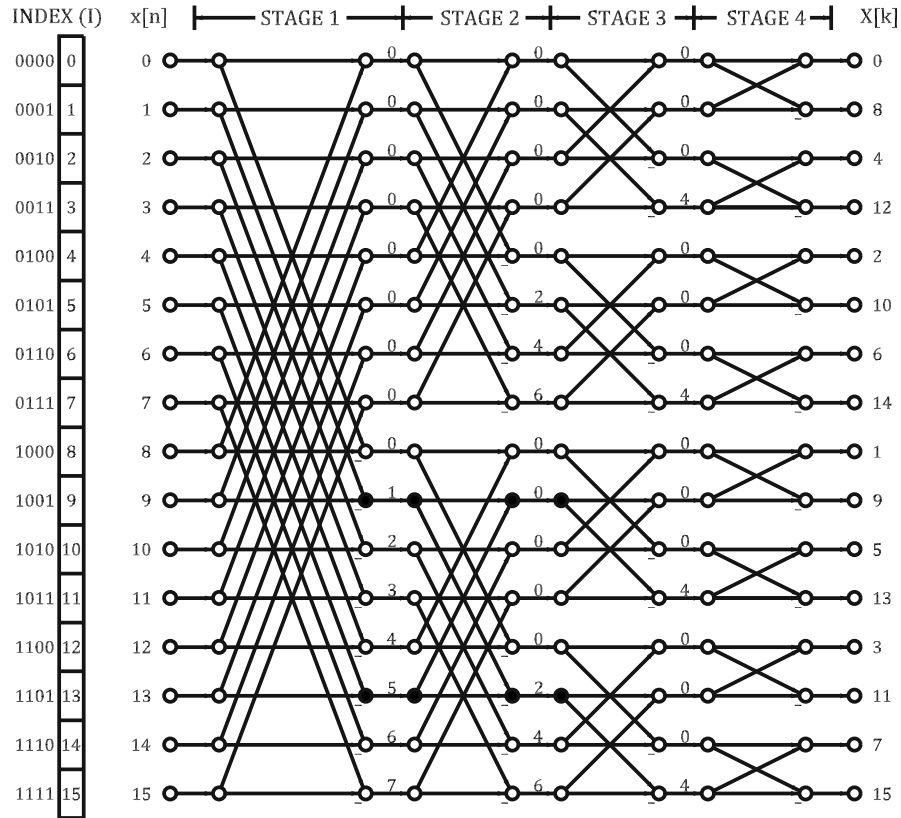
The algorithm can also be explained by looking at an example. A step by step modification of the butterfly highlighted using filled nodes in figure 2.9a is shown in figures 2.10, and the resulting twiddle factors are shown in figure 2.9b. Following is a brief description of the said modifications. The twiddle factor multiplication before the addition and subtraction is allowed to move to the inside of the butterfly, as shown in figure 2.10b. When the same factor is inside the butterfly on the both inputs to the adder, the multiplication can move further to the output of the addition. The same rules apply to the subtraction branch of the butterfly, and the resulting butterfly is shown in figure 2.10c. The multiplication can then be fused with the multiplication that was already there, by adding the exponents of the two twiddle factors, which results in twiddle factors shown in figure 2.10d.

It can be noticed that all of the twiddle factors in the odd stages have an exponent of $N/4$ after applying the transformation. The multiplication with such a twiddle factor is considered trivial as it can be implemented without a multiplier, by using the following transformation:

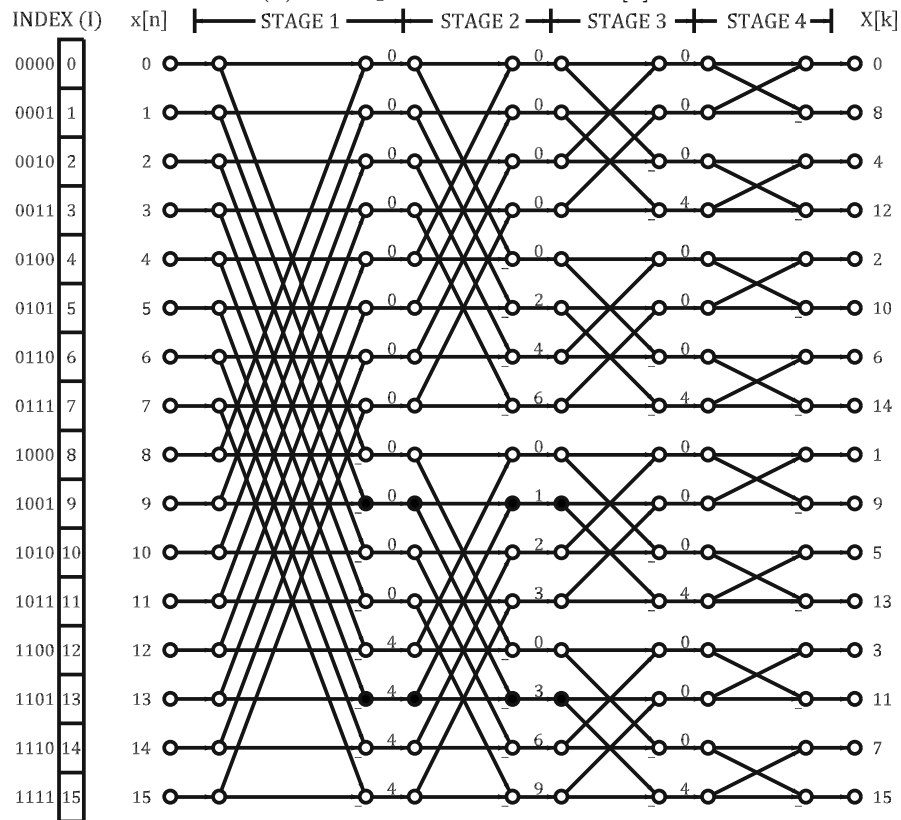
$$\begin{aligned}
 W_N^{N/4} &= e^{-j\frac{2\pi}{N} \frac{N}{4}} \\
 &= e^{-j\frac{\pi}{2}} \\
 &= -1j
 \end{aligned} \tag{2.16}$$

The rotation can then be implemented in the following way:

$$(a + j b) \cdot (-1j) = -j a + b = b - j a \tag{2.17}$$

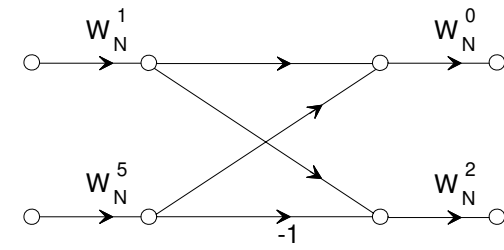


(a) A 16-point radix-2 FFT [9]

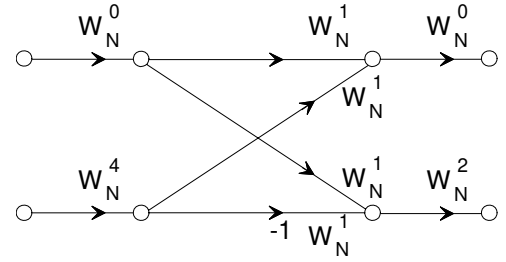


(b) A 16-point radix-2² FFT [9]

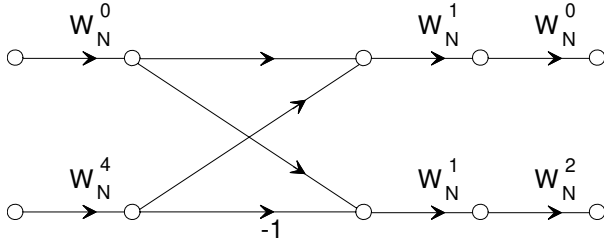
Figure 2.9: Radix-2² algorithm derived from radix-2



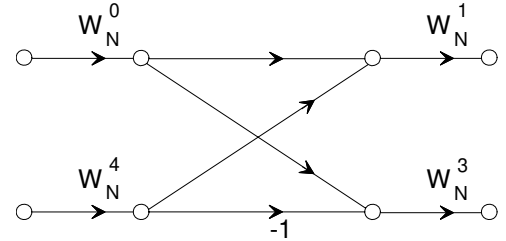
(a) The highlighted butterfly unit before modifications



(b) The highlighted butterfly unit after splitting twiddle factors



(c) The highlighted butterfly unit before merging the twiddle factors



(d) The highlighted butterfly unit after modifications

Figure 2.10: Merging two subsequent multiplications in radix-2² butterflies

It is shown that the complex multiplication with a trivial twiddle factor is realized by inverting the sign of the real part of the input, and swapping the real and imaginary parts.

When an FFT with an odd number of stages is implemented, the first stage must be implemented as a radix-2 stage, but the rest are allowed to be implemented using the radix-2² algorithm. Thus the complex rotators must support both radix-2² and radix-2 twiddle factors if a support for an odd number of stages is required.

When it comes to data order in radix-2² algorithms, the discussion on data order of a radix-2 FFT is relevant to radix-2² algorithms as well because the two only differ in some of the twiddle factor exponents. Additionally, the trivial multiplier stages will remain trivial after changing the data order as only the order of the twiddle factors changed.

2.4 Fixed-Point Arithmetic

To understand the need for fixed point arithmetic, it is necessary to discuss different representations of real value signals. Following is a brief description of fixed point arithmetics, while a much more thorough explanation can be found in [11].

Often in computing arises a need to represent real numbers, but computers can only represent numbers with finite precision. Integer values are commonly represented using two's complement, and real numbers are represented using floating point representation. Floating point representation is incredibly flexible and a great range of numbers can be represented with different resolutions, such that the relative quantization error is kept small. Such flexibility is good when dealing with numbers of unexpected range, but operations on floating point numbers are expensive due to a complicated binary representation consisting of three parts - a sign bit, a mantissa, and an exponent.

The fixed point representation is an alternative way of representing real numbers and does so by using integer numbers for the significant bits, and an implicit position of the binary point. That way, operations on fixed point numbers are as simple as integer operations, plus some additional effort to keep track of binary point position. The issue with fixed point numbers is their limited range which must be compensated by carefully determining the number of integer

bits to prevent overflow.

The number of integer bits (including the sign bit when using signed numbers) determines the range of numbers that is possible to represent using such a representation. Having too few integer bits can cause overflow to happen, which significantly impacts performance. The number of fractional bits determines the minimum difference between two numbers - resolution. If the number of fractional bits is too small, the quantization error becomes too high and degrades performance.

The resolution of the fixed-point representation corresponds to the value of the least significant bit, and is calculated using the following equation:

$$\Delta_{N_F} = 2^{-N_F} \quad (2.18)$$

where the number of fractional bits is denoted by N_F . The range of signed numbers that can be represented by a fixed-point representation is

$$\begin{aligned} n_{\max} &= (2^{N_I+N_F-1} - 1)/2^{N_F} = 2^{N_I-1} - \Delta_{N_F} \\ n_{\min} &= -2^{N_I+N_F-1}/2^{N_F} = -2^{N_I-1} \end{aligned} \quad (2.19)$$

where N_I and N_F are the integer and fractional number of bits in the corresponding fixed-point representation.

Implementation complexity is directly determined by the number of bits, therefore it should be kept as low as possible. This means having the lowest number of integer bits such that overflow does not happen, and the lowest number of fractional bits that has an acceptable quantization performance.

2.4.1 Rounding and Overflow

Other important parameters when it comes to fixed point operations are rounding mode and overflow mode.

Changing the number of bits in a fixed point number changes the exact values that are possible to represent using the new configuration. For instance, decreasing the number of integer bits decreases the maximum absolute value that can be represented.

Rounding and overflow modes determine what happens to the numbers that can no longer be represented.

Wrap and saturate are the most common ways of handling overflow. Wrap simply discards the selected number of most significant bits, leaving the others unchanged. This allows a number whose value can no longer be represented to wrap around to the values that can. The advantage of this method is the fact that it is easy to implement. The disadvantage however is the large discrepancy compared to the original number that can cause significant calculation errors. Alternatively, saturating the number to the maximum or minimum value (depending on the sign of the original number) that can be represented creates a discrepancy that is as small as possible. This however typically requires a two-step operation to be performed - first, the overflow must be detected and then the value of the number can be changed to the constant that corresponds to the maximum or minimum value in the new fixed point representation.

There are significantly more options for implementing rounding, each with its own strengths and weaknesses. The main goal of rounding is to create as little discrepancy as possible when discarding the least significant bits. This usually means rounding to the number that is found in the new fixed-point configuration and has the smallest difference to the original number. There are however several different options when the original number is exactly halfway between two numbers.

For instance, one can select the smaller of the two numbers. This method is known as "round-half-down". Similarly, selecting the larger of the two is known as "round-half-up".

While these methods implement rounding in a sense that the closest value is selected, both of them ultimately have a bias toward smaller/larger values. This bias can only be observed when the original number is exactly at the midpoint between two numbers. The other issue is the asymmetrical behaviour of the positive and negative numbers. In the case of "round-half-up", the positive values are biased toward larger absolute values, while the negative numbers are biased toward a smaller absolute value.

As a solution for the two problems, rounding toward zero and rounding from zero are possible. While these two methods do not suffer from the bias toward larger or smaller values, they create a bias in the absolute value of the signal. Rounding toward zero happens to slightly decrease the amplitude of the signals after rounding, while the rounding from zero does the opposite. Additionally, if unsigned numbers are used, these methods do not differ from the previous two.

Another way of resolving the halfway point is to select the even/odd of the two numbers. Evenness is determined by the number's least significant bit. These methods are commonly known as "round-half-to-even" and "round-half-to-odd".

The rounding methods described so far are all considered deterministic because the result is determined by the input number only. An alternative way of choosing between the two possible values is using an additional stochastic variable. Statistically, this method does not exhibit any bias and is fair toward both even and odd values.

In an FFT, the quantization error is introduced only in the complex multiplications. Trivial rotations explained in previous section do not insert any error because they are implemented without any mathematical operations, and without changing the data width. Additions, while subject to overflow, also do not insert any quantization error because the number of fractional bits remains the same after the addition and thus no rounding is needed. Thus, in the FFT, rounding is considered only for the complex rotators, while overflow must be prevented in the adders.

Preventing width growth caused by mathematical operations can be done in several ways, two of which are explored in [12].

The first strategy aims to keep the number of fractional bits constant in all of the stages, while the number of integer bits grows to prevent overflow when doing the additions. It was shown that an increase of 1 integer bit per stage is sufficient at preventing overflow. The second strategy keeps the total number of bits constant across stages. This means that the increase of the number of integer bits is done at the expense of decreasing the number of fractional bits. This causes round-off error which should be minimized by implementing rounding. This data shift can be implemented at the output of the complex multipliers, where rounding is already implemented as a means to lower the rounding error when dropping the least significant bits at the output of the multiplier.

When implemented using the second strategy, doubling the FFT size causes a 3dB drop in SQNR performance; while the SQNR performance of an FFT implemented using the first strategy does not depend on the FFT size.

Chapter 3

Software Reference Model

Before diving into the FPGA hardware implementation, it is beneficial to first develop a software model that enables us to evaluate the operations that hardware is supposed to perform, and eventually determine parameters such as the numbers of integer and fractional bits in the fixed point representation. This allows a timely fine-tuning of the parameters, and also attempts to locate any issues that could pose a problem in the hardware implementation.

The software model is written in the Python programming language, using a custom library that supports fixed-point arithmetic. A brief overview of the library is given, but using any library that supports fixed-point arithmetic is possible.

The library is implemented by deriving a class from the numpy array [13] and reimplementing operations such as addition, subtraction, and multiplication, whereas division is not needed.

The class contains two arrays of signed 64-bit integers: one for the real part and the other for the imaginary. The numbers of integer and fractional bits must also be stored. The main part of the new class however are the functions that operate on the fixed-point array. This includes operations between two fixed-point arrays, but also the operations that change the fixed-point parameters.

The array constructor takes the necessary fixed-point parameters, and the array from which to initialize the data. Data can be initialized using a floating point array, which is then converted to the fixed-point representation with the supplied parameters. The implemented rounding modes are truncation, "round-half-up" and "round-half-to-even". Wrap and saturate are the available options of handling overflow.

3.1 Architecture

The software model of the FFT follows the principles of Object-Oriented Programming and uses classes to divide components into separate and isolated units. This has the benefit of making a distinction of what happens at runtime, and what happens at compile time. Runtime parameters are implemented as setters and getters, while the compile time parameters cannot be changed after the class instance has been created.

For instance, as the FFT length is fixed and twiddle factors are thus constant, twiddle factors can be stored in a read-only memory, calculated during compile time. If non-constant twiddle factors are required (such as for the ability to change FFT length in runtime), they have to be stored in RAM, and a method of calculating them should be implemented, typically implemented using the CORDIC algorithm.

Each butterfly as per the description in the previous chapter is here divided into a pre-adder and a rotator. The pre-adder forms an output array by doing the additions and subtractions of the input data, while the rotator does the multiplications with the twiddle factors.

The software model consists of an array of pre-adder units, and an array of rotators. The operations on the input array are done in a `for` loop over the number of stages that exist for the specified FFT configuration.

All of the components' classes have a function called `step` that does the processing of the input array and returns the output. This is done to create a sort of a streaming interface where the data flows through components from the input to the output. This is shown in figure 3.1.

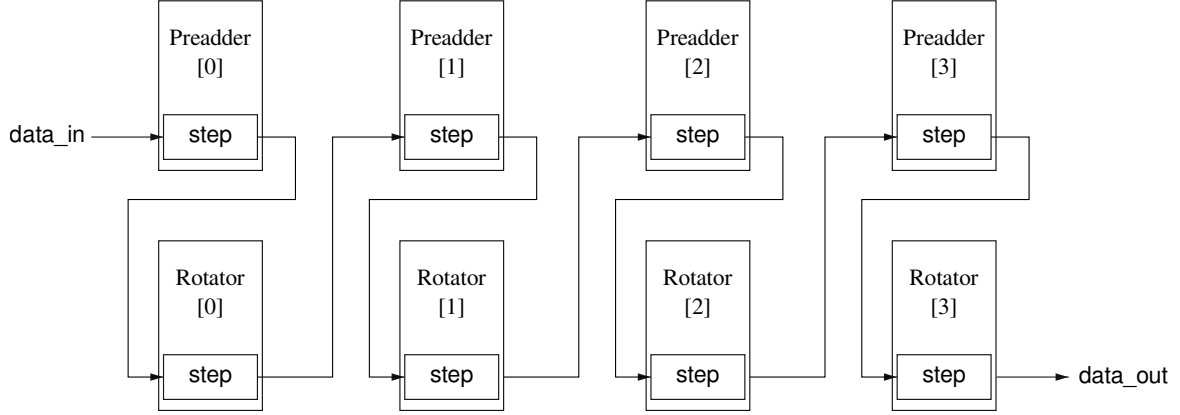


Figure 3.1: Software model architecture

3.1.1 Pre-adder

The pre-adder is the component that implements the additions and subtractions according to the signal flow diagram shown in figure 2.3. Creating a `Preadder` object requires three parameters: the stage index, the number of stages in the FFT, and the data order. The component then calculates the butterfly size using the following formula:

$$N_A = \begin{cases} 2^{m-s}, & \text{natural input data order,} \\ 2^{s-1}, & \text{bit - reverse input data order} \end{cases} \quad (3.1)$$

where s represents the stage index, while m is the total number of FFT stages.

The butterfly size determines the pattern of additions and subtractions. It represents the number of consecutive addition outputs, which are followed by the same number of the outputs of subtraction. The output sample $y[k]$ can be calculated from the input array using the following rule:

$$y[k] = \begin{cases} x[k] + x[k + N_A], & k \bmod (2 N_A) < N_A, \\ x[k - N_A] - x[k], & \text{otherwise} \end{cases} \quad (3.2)$$

This is implemented by reshaping the input array into a two-dimensional numpy array, whose number of columns should be equal to the butterfly size. Then, every even-numbered row is summed with the row that follows it to calculate the addition outputs that will be stored in the even-numbered rows of the output array. Similarly, to calculate the odd-numbered rows of the output array, the odd-numbered rows should be subtracted from the even-numbered ones. The output should then be flattened to a one-dimensional array.

If a fixed-point model is used, both the input data width and the number of fractional bits should be preserved. Preserving the number of integer bits leads to a potential overflow, which would introduce significant errors that would decrease the SQNR performance. Overflow should be prevented by making sure the input data of the pre-adder has a sufficient number of integer bits.

3.1.2 Rotator

A complex rotator is an element used for multiplying data with twiddle factors. As the modulo of twiddle factors is exactly 1, this multiplication can be represented by a rotation in the complex plane.

First, twiddle factors are generated, and then optionally converted to a fixed point representation and stored as a constant array. As the twiddle factors are on the unit circle by definition, the maximum absolute value of their real and imaginary parts is also 1. Taking the sign into account causes the number of integer bits of twiddle factors to be exactly 2.

Calling the rotator's **step** function does a complex multiplication of input data and the pre-calculated twiddle factors. If a fixed-point calculation is required, the output should then be rounded to the specified number of fractional bits according to the selected rounding method.

Chapter 4

Hardware Architecture

A pipelined hardware architecture of FFT consists of m stages, and data is buffered after every stage.

Each stage in a fully parallel architecture has N inputs and N outputs and processes N samples in every clock period. This achieves the maximum possible throughput of N samples per clock period, and it is implemented by having N complex adders and up to $N/2 - 1$ complex multipliers for the samples that have non-unity twiddle factors.

The first problem of such an architecture is the required hardware utilization - such a large number of adders and multipliers is infeasible, or even impossible to implement for large enough FFT sizes. The second problem is the fact that all of the samples are required at the same time, requiring enough memory to hold N samples in each of the stages.

Hardware utilization of FPGA resources refers to logic, DSP and memory resources; as they are the most used. Logic (FPGA LUTs) along with carry chains are used to implement addition and subtraction. Multiplication can be implemented in logic, but most modern FPGA platforms have some kind of integrated DSP units that have a hardware multiplier. There are several types of memory present in modern FPGA platforms that target different use cases, and that perform very differently - there is the distributed RAM that is mostly used for small buffering, and block RAM that is used when a much deeper memory is required.

4.1 Pipelined Streaming FFT Architectures

Streaming architectures try to solve the problem of a high resource usage by sequentially processing the samples. Streaming architectures commonly process only one input sample in a clock period, but there are architectures that have larger data parallelism. Streaming architectures reuse one adder/subtractor pair and one multiplier per stage to process different data samples sequentially. Memory buffer in every stage is also required, but it has a lower depth compared to that of the fully parallel implementations.

There are numerous different streaming architecture designs with different memory and logic requirements. Therefore, depending on the required memory and logic utilization, different architectures can be implemented. Single-path Delay Feedback (SDF) and Single-path Feed-forward (SFF) are described below as single-path streaming architectures, and Multi-path Delay Feedback (MDF) is selected as a multi-path streaming architecture that is also described. A comparison of a much larger number of architectures can be found in [9].

Comparing SDF architecture to SFF, it can be concluded that the SFF architecture has a lower number of adders, but requires more memory. In this master's thesis, only SDF and MDF architectures were implemented, but an overview of the SFF architecture is also given.

4.1.1 SDF

Single-path Delay Feedback (SDF) architecture is one of the earliest pipelined streaming FFT architectures, that appeared in [14]. In the following paragraphs is a brief overview and explanation of the architecture.

SDF FFT consists of m stages, each with one input and one output. Data order at the input of each stage is the same as the order in the signal flow graph from top to bottom. A diagram of SDF FFT is shown in figure 4.1. In it, stages are shown consisting of a pre-adder (representing the additions and subtractions in the butterfly units), and a rotator that does the multiplication of samples with twiddle factors. Note that a non-trivial rotator is represented by a circle, and a trivial one is diamond-shaped.

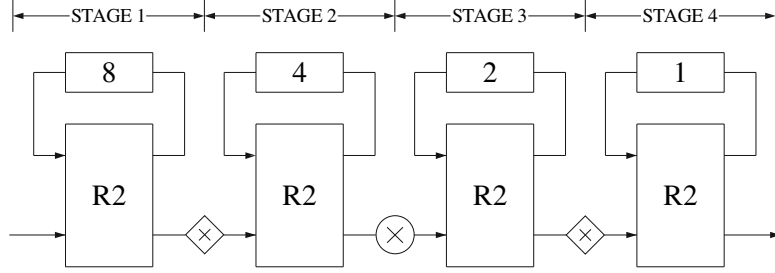


Figure 4.1: Radix-2 SDF FFT architecture [9]

Non-trivial rotators in an SDF stage are simple - the output of the pre-adder should be multiplied by twiddle factors sequentially, and twiddle factors can reside in Read-only memory (ROM) that is sequentially and circularly accessed. Trivial rotators are even simpler - they do not need additional twiddle factor memory, and only have a very regular sequence of flags that determine whether to do a -90° rotation or not.

SDF pre-adder takes a stream of samples at the input and does the addition/subtraction corresponding to the butterfly size as shown in the signal flow graph. Figure 4.2 shows data at the input of the stage as one stream, and the other stream consists of the same input samples, only delayed such that the elements are correctly lined up for addition/subtraction.

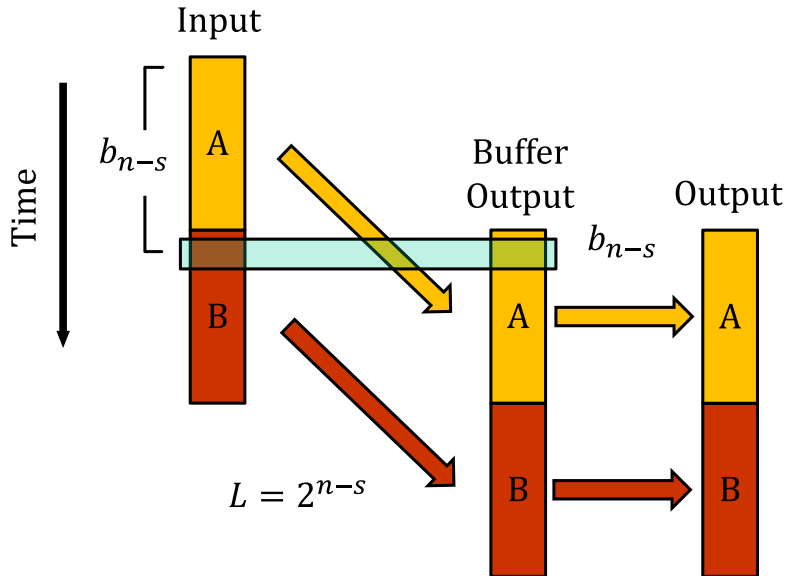


Figure 4.2: Signal flow in an SDF stage [9]

A FIFO buffer of depth L is used to align the incoming data samples. When the corresponding samples are aligned, their addition and subtraction should be performed in parallel.

The result of the addition can be immediately streamed to the output, while the result of the subtraction should be buffered, and streamed after all the additions have been sent to the output. For this, an additional FIFO buffer can be used, and we get a structure shown in figure 4.3.

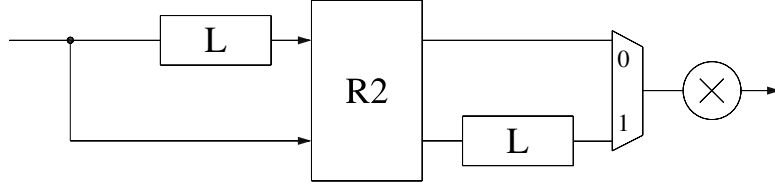


Figure 4.3: A memory inefficient FFT stage architecture

However, it can be noticed that each time a sample from the input FIFO is consumed, data for the output FIFO is generated, and the overall number of samples stored in the FIFOs remains the same. It is therefore possible to reuse the same FIFO to first buffer input samples, and then the results of the subtraction while the input samples get consumed. This leads to a decrease in memory usage and an architecture whose diagram is shown in diagram 4.4, but this requires a multiplexer at the input of the FIFO and a multiplexer at the output.

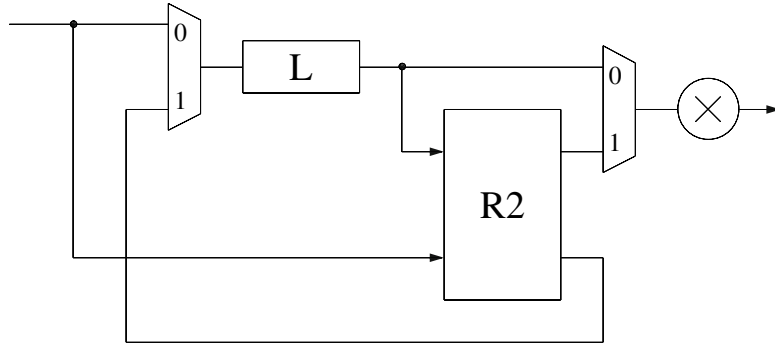


Figure 4.4: SDF stage architecture [9]

The depth of the FIFO buffer varies among the FFT stages and is equal to the butterfly size. Memory required for the whole FFT can be calculated by summing the FIFO depths of all the stages:

$$R_M = \sum_{i=0}^{m-1} 2^{m-i} = 2^m - 1 = N - 1 \quad (4.1)$$

The number of adders in each stage is exactly 1, therefore the adder complexity of FFT composed of SDF stages is proportional to m . If the total number of stages is even, the number of non-trivial rotators is equal to $\frac{m}{2} - 1$. Otherwise, the number of non-trivial rotators is equal to $\frac{m-1}{2} = \frac{m+1}{2} - 1$.

$$R_D = \left\lceil \frac{m}{2} \right\rceil - 1 \quad (4.2)$$

As per the discussion on data order in the previous sections, changing the input data order causes the butterfly sizes to change, and twiddle factor order to change. The overall resource utilization remains the same. This leads to an FFT architecture that is shown in figure 4.5.

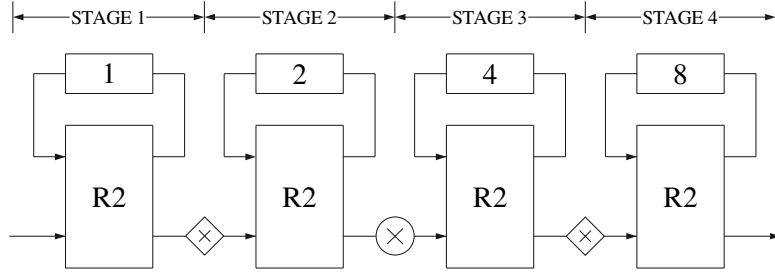


Figure 4.5: SDF FFT with a bit-reverse input order [9]

4.1.2 SFF

Similar to the SDF architecture, buffering is done to align data samples at the input of a stage, but instead of calculating both the addition and the subtraction in parallel, only one of those operations is done at any given moment. That makes it possible to reuse the same hardware that does the addition to do the subtraction, and directly stream the adder output to the rotator. However, as both operands are required to do the subtraction, both data samples have to be buffered until all of the additions are performed. This requires a second FIFO buffer of the same depth.

This produces the SFF data flow diagram with the same format as was shown for SDF architecture, shown in figure 4.6.

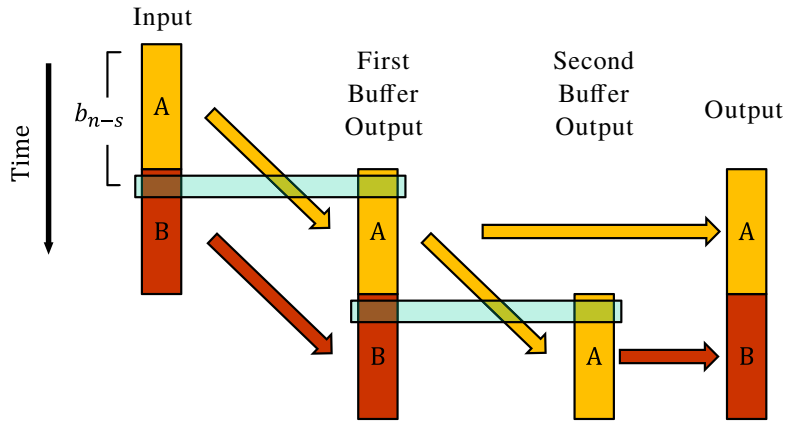


Figure 4.6: Data flow of an SFF stage

SFF stage architecture thus has two memory buffers and a multiplexer, as shown in figure 4.7.

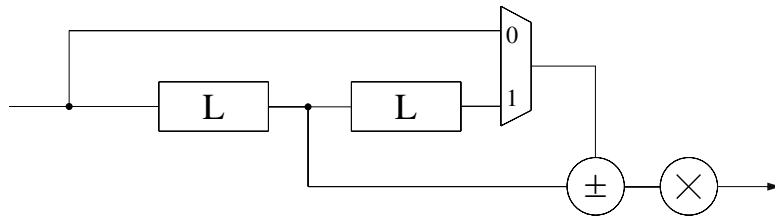


Figure 4.7: SFF stage architecture

Although the memory requirements are larger, this is partly amortized by some features of the contemporary Xilinx FPGA architectures, namely shift registers. For instance, two FIFOs

of the same depth can be implemented in the same amount of hardware resources necessary for one such FIFO if the depth less than or equal to 16 [15].

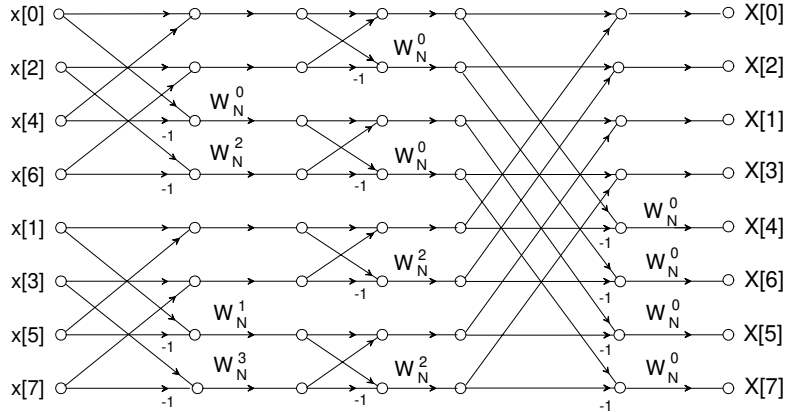
The additional benefit of having a single addition before a rotator is the ability to integrate both the addition and the subtraction into the DSP blocks that do the complex multiplication, if such a multiplication follows. The number of small depth FIFOs that are followed by a non-trivial multiplication is small, but the gains may be valuable. Consequently, the number of such cases is slightly larger if the input order is set to bit-reverse. Other than that, changing the input data order has the same implications that stood for the SDF architecture.

The SFF architecture stages do not have to exist exclusively in an FFT, but they can be used interchangeably with the SDF stages. This allows implementing SFF stages in the stages where data buffering requirements are small, and SDF architecture for the other FFT stages.

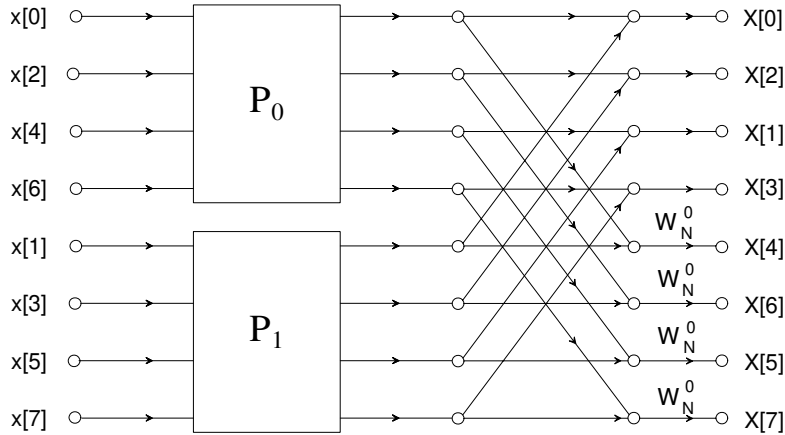
4.1.3 MDF

Multi-path Delay Feedback (MDF) is an architecture that is very similar to SDF, and most of the design units remain the same, but it does processing of multiple data samples in parallel. It can be derived from the SDF architecture by reordering the signal flow diagram and dividing it into several distinct units. An example follows.

If an FFT with a parallelism of P is required, rows in the signal flow graph should be reordered into P groups ($P_i, i \in 0, \dots, P-1$) in such a way that all the rows in the group P_j have the remainder of j when divided by P . This grouping causes the first $m-q$ butterfly units near the input to become smaller, and the last q butterfly units to grow, where q denotes the expression $q = \log_2(P)$. An example of this for $P = 2$ and $N = 8$ is given in figure 4.8a.



(a) Data flow graph used to derive the MDF architecture



(b) Data flow graph used to derive the MDF architecture

Looking at the first $m-q$ stages, it can be seen that each group forms a structure similar to

an N/P -point FFT (differing only in twiddle factor exponents), and those can be implemented as SDF stages with modified rotators. The P groups should coexist and operate in parallel, each group processing a different sample. Their outputs should then be combined in butterfly units according to the signal flow diagram. This is represented by a diagram shown in figure 4.9.

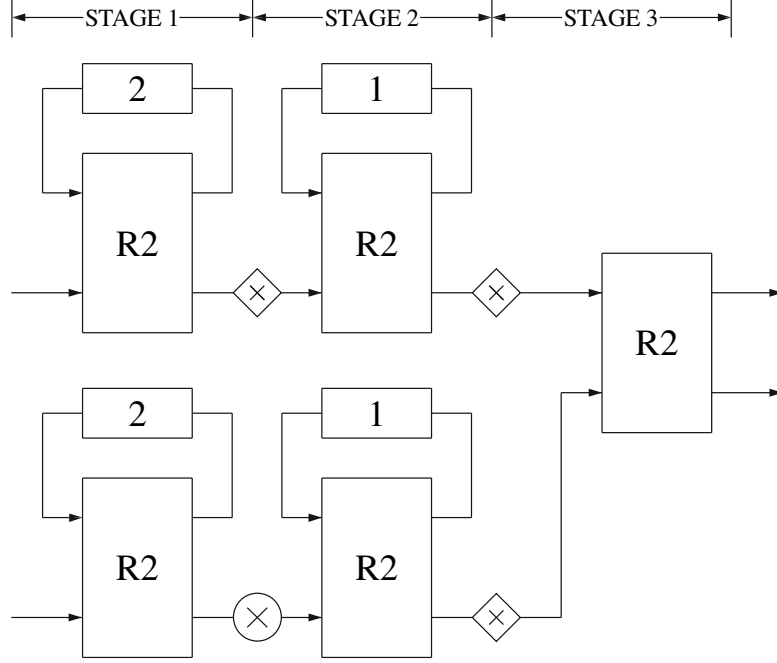


Figure 4.9: MDF FFT architecture [9]

A rotator slightly different from the one used for the SDF architecture is required. Grouping of the input samples results in a rotator being split into P different rotators, each with a subset of the original twiddle factors. The division of the twiddle factors among P rotators is done in such a way that the rotator in the stage s in a path with the index p contains the following twiddle factors:

$$w_{s,p}[n] = w_s[n \cdot P + p] \quad (4.3)$$

where $w_s[k]$ denotes the array of twiddle factors in the stage with the index s , and n is in the following range $n \in 0, \dots, N/P - 1$.

Note that the splitting the twiddle factor array into P subarrays may create a rotator whose twiddle factors can be implemented without a multiplier. However, delay lines must then be implemented to make sure the data stays aligned in all of the paths.

Similarly, if an FFT with bit-reverse input order is required, the same operations should be performed on the signal flow diagram shown in figure 2.3 - the difference is that this time row numbering is done according to the output sample index. This results in a signal flow diagram shown in figure 4.10 using $P = 2$ and $N = 8$ as an example. As expected, the graph is very similar to the graph used when looking at natural order input MDF architecture, the sizes of the butterfly units got reversed. The resulting architecture is shown in figure 4.11.

The resulting architecture has a larger adder and multiplier complexity compared to the SDF, because P operations must be done in parallel in every stage. That means the number of complex adders and multipliers is P times larger compared to the number in an SDF architecture FFT. The memory requirements however have decreased slightly.

The number of samples that need to be buffered can be calculated using the following expression:

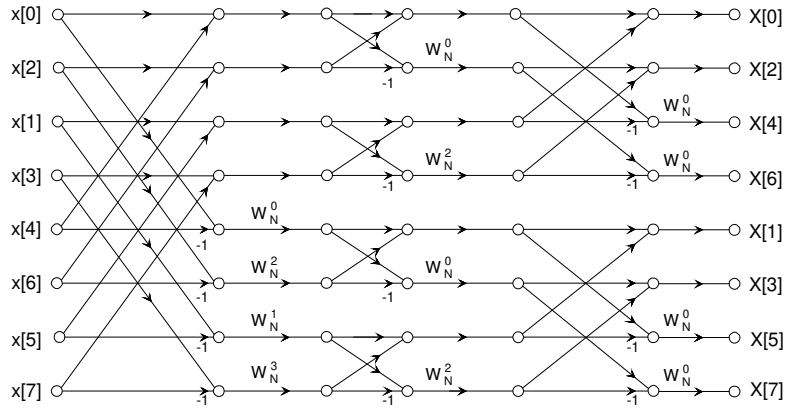


Figure 4.10: Data flow graph used to derive MDF architecture with bit-reverse input order

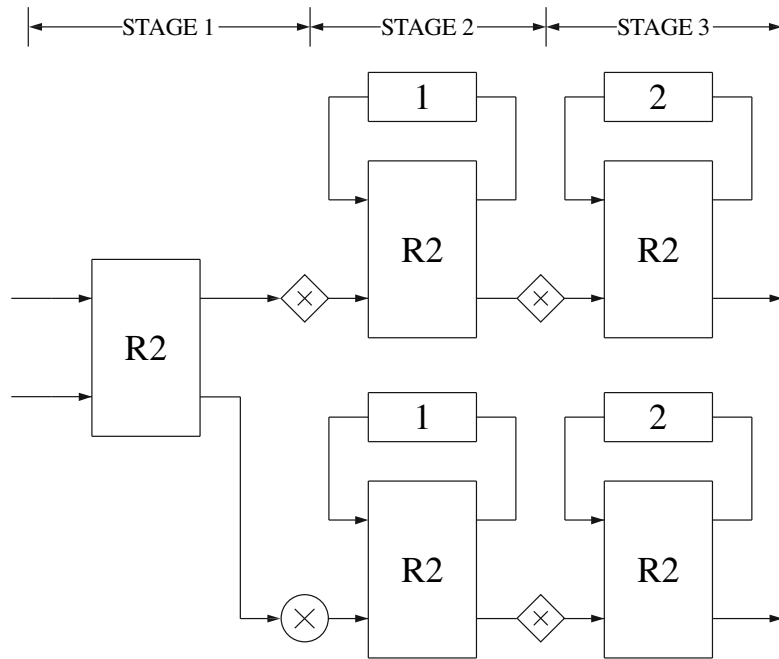


Figure 4.11: MDF FFT architecture with bit-reverse input order

$$R_M = P \cdot \left(\sum_{i=0}^{m-q} 2^i \right) = P \cdot (2^{m-q} - 1) = P \cdot \left(\frac{N}{P} - 1 \right) = N - P \quad (4.4)$$

where N denotes the FFT size, while P is the selected parallelism.

4.2 Data Reordering Circuit

As discussed previously, the DIF and DIT algorithms change the order of data when applied to an array of samples. The radix 2 DIF algorithm has natural input data order and bit-reversed output data order, but can be transformed so that the input data order is bit-reversed and the output natural. For streaming FFT architectures, this determines the order in which the data samples appear one after another. If it is required for both input and output data order to be natural, one can implement a data-reordering circuit after a regular DIT/DIF based FFT algorithm.

Even though the circuits were not implemented, a discussion of architectures is given in the following pages for completeness.

4.2.1 Serial Pipelined Architectures

Presume that the input data stream consists of integers in the range from 0 to $N - 1$, and the goal is to make an output stream that changes the order of the samples such that the numbers appear in bit-reverse order at the output.

It is obvious that a number of samples must be buffered, but the number of samples to buffer is a little harder to determine.

The first solution is an N samples deep memory buffer that is written to in natural and read out in bit-reverse order. No writing occurs during the readout and vice-versa. This is mandatory in order to prevent overwriting data in memory. The disadvantage of such approach is that the data flow must be stopped for half the time to read the buffer.

That can be easily solved by double buffering - having two identical memories that operate at different times. Both buffers start as empty, and the data is first written to the first buffer. Because the second buffer is empty, no data is output. After the first buffer fills, data can be streamed out of the first buffer in bit-reverse order. At the same time, the input samples are being written to the second buffer. After the first buffer is completely empty and the second buffer completely full, input is once again streamed into the first buffer. At the same time, samples from the second buffer are streamed to the output. Such a solution achieves maximum throughput, but requires two buffers with N samples each.

If changing the order from natural to bit-reverse is viewed as an operation, it has the property of self-inversion. That is, applying the operation twice on an input array yields the input array itself. That is an important property for the architecture of the final bit-reversal circuit.

This architecture consists of a memory buffer that stores N points. Similar to the previous two architectures, the buffer starts out empty and the input data is first written to the memory in the natural order. After the memory is completely filled, the samples can be read out of the memory in bit-reverse order. Note that each time a sample is read out of the memory, the memory slot it previously occupied is free and can be written safely. Thanks to this, the samples from the input stream can now be written to the same buffer, but this time on memory locations that are in bit-reverse order. When all of the first N points are streamed at the output, the second group of N points occupies the memory. Since they were written to the memory at addresses that are in bit-reverse order, the memory needs to be read out

in a natural sequence to perform the reordering. Once again, a sample from the input stream is allowed to be written to the memory each time a sample is read from the memory to the output, this time with addresses in natural order.

It can be seen that, thanks to the property of self-inversion, memory requirements for a data reordering circuit can be halved.

4.2.2 Parallel Pipelined Architectures

Parallel streaming FFT implementations derived from DIT or DIF algorithm also suffer from the problem of data order - if input data order is natural, the output must be in bit-reverse order; and vice versa.

Only P -parallel architectures are discussed, such that P is a power of two; and only radix-2 bit-reverse order is considered. Consider a stream of N samples, and P samples are streamed in parallel. This requires exactly $B = N/P$ stream beats to complete. It is considered that the input data arrives in bit-reverse order, while the output of the circuit should be in natural order.

The data that arrive at the path with index p , where $p \in 0, \dots, P-1$, is given using the following formula:

$$X_p[t] = X[BR(tP + p)] \quad (4.5)$$

$BR(k)$ represents the function that performs bit-reversal of the sample index k , and k is represented using an m -bit unsigned integer; while t is a time index in the range $t \in 0, \dots, N/P - 1$.

Similarly, the output of the path number p of the bit-reversal circuit should be $Y_p[t]$, and is defined using the following expression:

$$Y_p[t] = X[tP + p] \quad (4.6)$$

It should be noted that the samples that the lowest q bits of the output sample number determine the path index at the output of the bit-reverser circuit, while the rest of the bits determine the time index when the sample should be output. Because of the bit-reversal of the sample index, this means that the q most significant bits of the index at the input of the bit-reverser determine the path index of the output [16]. The m -bit representation of the sample index k is shown in figure 4.12.

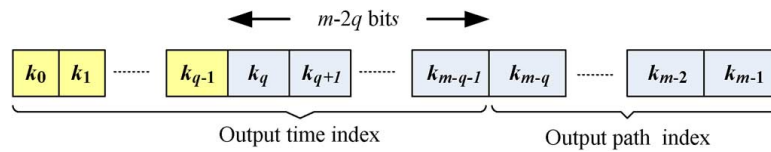


Figure 4.12: Binary representation of the sample index used for parallel bit-reversal [16]

Because P output samples are required at any given moment, P memory buffers are necessary, and the P output samples should be written to different memory buffers. This is done to avoid conflicts where more than one sample is required from a memory buffer. Thus, the q most significant bits of the input sample index determine the memory buffer where the sample should be stored. Thus, [16] proposes an architecture with write and read commutators connected to the input and output of the memory buffers, respectively. This is shown in figure 4.13.

The architecture achieves a throughput of P samples per clock cycle, and requires a buffering of N data samples divided into $2P$ banks.

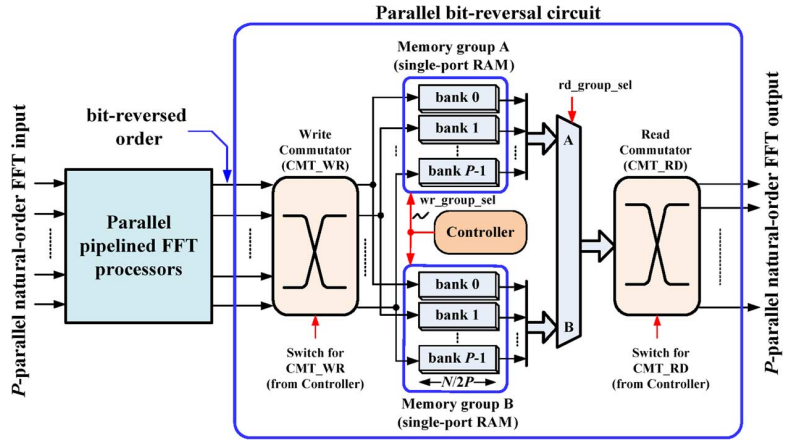


Figure 4.13: Parallel bit-reversal circuit [16]

Chapter 5

Hardware Implementation

Following is a description of the implementation of a DIF radix-2² FFT. The FFT is implemented in VHDL, and synthesized using Xilinx Vivado 2021.1 for a device from Zynq Ultrascale+ family. The DIF algorithm was chosen as its butterfly geometry allowed for more hardware optimizations compared to the DIT algorithm. The radix-2² was chosen for its low multiplier complexity compared to the radix-2 algorithm.

The AXI4-Stream protocol was chosen as an interface of the components because of its low wire count and high performance. The bus is directed - the master is the side that initiates the data transfer, while the slave only receives the data. Both sides are allowed to stall the transaction if the component is unable to process the transfer at the moment. The data stream may be divided into bursts divided using a dedicated signal called `last`.

Using a single standard protocol significantly simplified the testing and verification of the components. All of the components have a testbench written in VHDL that compares the results of the simulation to the software reference model. The simulation was performed using Xilinx Vivado 2021.1 [17].

Following is a description of the signals that comprise the AXI4-Stream bus that were used, while a more detailed explanation of the AXI4-Stream protocol can be found in the AMBA standard [18].

Table 5.1: An overview of the signals in the AXI4-Stream protocol

Signal name	Mandatory	Driver	Description
<code>data</code>	yes	Master	The data to be transferred
<code>valid</code>	yes	Master	Denotes the data on the <code>data</code> bus is valid and should be transferred
<code>ready</code>	yes	Slave	Denotes the slave is ready to receive the data present on the <code>data</code> bus
<code>last</code>	no	Master	Denotes that there is no more data in the current burst

5.1 Stage-chain

FFT can be split into several stages, each of them independent. Each of the stages has a slave AXI4-Stream port on the input and a master AXI4-Stream port on the output. Connecting the different stages together is easy, as the input protocol of a stage is the same as the output protocol of the stage before it. Diagram 5.1 shows different stages along with interface wires between them.

Each stage is given a unique index that is also shown in the diagram.

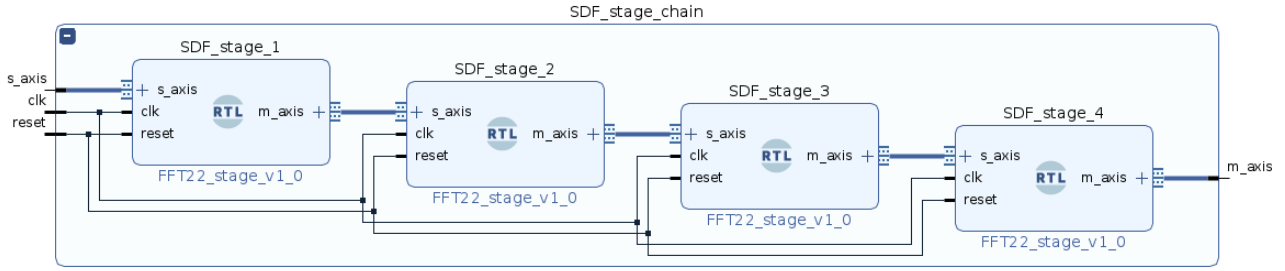


Figure 5.1: An FFT composed of 4 stages

5.2 Pre-adder

A pre-adder has to buffer the input stream in a FIFO in order to align the data that are to be added together. A diagram of the pre-adder is shown in figure 4.4. The counter for controlling the multiplexers and FIFO depth are determined by the stage index and the FFT size.

The FIFO buffer is used for two functions - the first is to buffer the input samples in order to align the samples that are to be added together, while the second is to buffer the results of the subtractions until it is their order to be streamed at the output. That requires a multiplexer at the input to the FIFO, with the two inputs being the input stream and the subtraction output of the butterfly.

The output stream comes from two sources - the first is the addition output of the butterfly, and the second is the output of the FIFO. That requires a multiplexer at the output of the pre-adder, with the two inputs being the addition output of the butterfly and the FIFO output.

The control signals for the multiplexers are generated from data sample counters present at the output of the multiplexers.

Following is a description of the function of the blocks that make up a pre-adder, and a mapping to FPGA resources is also presented.

5.2.1 FIFO

A First-In-First-Out (FIFO) buffer is a type of memory where the order of the data at the input of the memory, and the output of the memory are the same. Following is the implementation of a FIFO buffer with AXI4-Stream interfaces at the input and output. Two address registers are shown - read address register, and write address register. The RTL code implements a simple dual-port RAM memory, with additional logic for control signals that handle AXI4-Stream protocol.

The implementation targets memory resources found in the Xilinx Ultrascale+ hardware architecture, and especially for Block RAM [19]. As such, some design decisions were made to enable the FIFO to take as few resources as possible, while able to be implemented at a very high clock rate (greater than 400 MHz). This includes output registers to achieve lower clock-to-out delay of the FIFO, shown in figure 5.2.

Block RAM is only suitable for implementing large-depth FIFOs. If the depth of FIFO is less than 512 words, a unit of block RAM is only partially utilized, thereby wasting resources.

For such cases, an implementation in other types of memory, such as distributed RAM or shift registers, is preferred. A detailed explanation of the hardware required for this can be found in [20].

For implementing a FIFO in distributed RAM, control signals such as a read and write pointer and read and write enable signals have to be generated, while the data is written to the distributed RAM instance in a synchronous manner.

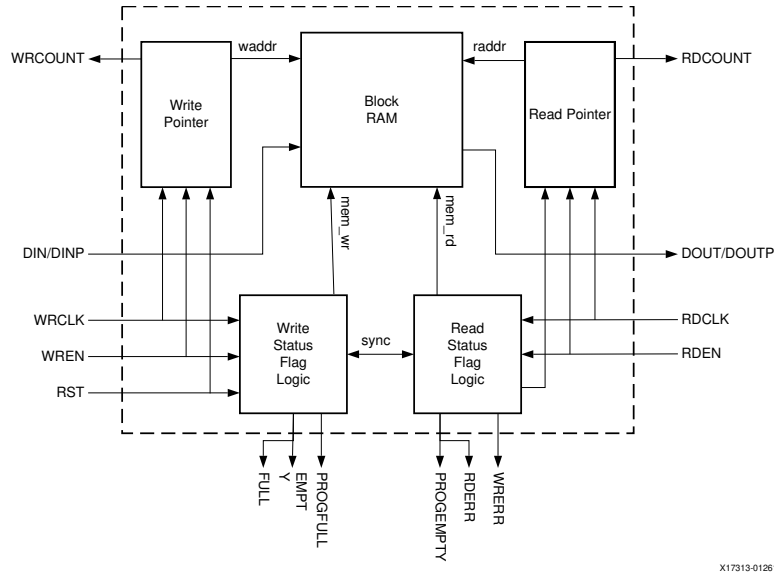


Figure 5.2: Architecture of the block RAM FIFO in Ultrascale+ architecture [19].

There are also FIFO memory implementations available from FPGA vendors, notably the AXI-Stream data FIFO from AMD/Xilinx [21], and a FIFO IP from Intel/Altera [22]. The FIFO IP from Altera does not use an AXI4-Stream interface, but creating a shim to convert the IP's interface to an AXI4-Stream is relatively simple.

It should be noted that the block RAM tiles include a built-in FIFO support, but such hardware implementation requires using a vendor-specific instantiation template which limits code portability.

5.2.2 Butterfly

A butterfly unit is a component with two input ports, and two output ports. The two inputs represent the arguments for the two outputs - addition and subtraction. All of the ports are using the AXI4-Stream interface. This slightly increases the logic utilization because of the AXI4-Stream handshake, but having compliant interfaces with other components is beneficial.

5.3 Trivial Rotator

A trivial rotator is a rotator that only does an optional complex rotation by -90° . This is done according to the radix-2² signal flow graph shown in figure 2.9b. It can be noticed that the rotations by -90° are present only in the last quarter of the sample indices.

If a trivial rotator is represented by a flag generator and a multiplexer, the multiplexer is used to either propagate unchanged input sample or a rotated one based on the output of the flag generator. The flag generator can thus be a counter whose two most significant bits determine the output.

If bit-reversed data order is generated, the counter should also count in bit-reverse order.

5.4 Complex Rotator

A complex rotator is a component dedicated to multiplying the input samples with a specific sequence of twiddle factors. This requires a twiddle generator - the component that provides

the twiddle factors; and a complex multiplier - dedicated to multiplying the samples with the twiddle factors.

The twiddle factors are usually stored in a Read Only Memory (ROM), but this may be suboptimal for larger FFT sizes. In that case, a twiddle factor generator with the CORDIC unit may be preferable.

If a ROM implementation is used, it may not be necessary to store all the twiddle factors in the memory. As a reminder, twiddle factors are defined using the following formula:

$$W_N^n = e^{-j\frac{2n\pi}{N}} = \cos\left(\frac{2n\pi}{N}\right) - 1j \cdot \sin\left(\frac{2n\pi}{N}\right) \quad (5.1)$$

where $n \in 0, \dots, N-1$.

This is an array of N complex integers, shown in the complex plane in the figure 5.3. From the definition itself, it can be noticed that the array represents N uniformly distributed points on a unit circle in the complex plane.

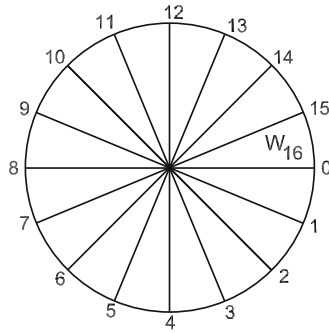


Figure 5.3: Twiddle factors on a unit circle [23]

It was shown that the regular structure of the twiddle factors makes it possible to decrease the amount of memory required for storing the twiddle factor values. This is done by storing a fraction of the points and using mathematical transformations to calculate the rest. For instance, Hasan [24] proposed that only $N/8 + 1$ points are required, while the rest can be calculated by changing the sign and/or exchanging the real and imaginary parts.

Therefore, if C_n represents a twiddle factor to the power of n , $C_n = W_N^n$, then the value of the twiddle factors in the different octants can be represented by the following identities [25]:

$$W_N^n = \begin{cases} C_n & 0 \leq n \leq \frac{N_{max}}{8}, \\ -j C_{\frac{2N_{max}}{8}-n}^* & \frac{N_{max}}{8} < n \leq \frac{2N_{max}}{8}, \\ -j C_{n-\frac{2N_{max}}{8}} & \frac{2N_{max}}{8} < n \leq \frac{3N_{max}}{8}, \\ -C_{\frac{4N_{max}}{8}-n}^* & \frac{3N_{max}}{8} < n \leq \frac{4N_{max}}{8}, \\ -C_{n-\frac{4N_{max}}{8}} & \frac{4N_{max}}{8} < n \leq \frac{5N_{max}}{8}, \\ j C_{\frac{5N_{max}}{8}-n}^* & \frac{5N_{max}}{8} < n \leq \frac{6N_{max}}{8}, \\ j C_{n-\frac{6N_{max}}{8}} & \frac{6N_{max}}{8} < n \leq \frac{7N_{max}}{8}, \\ C_{\frac{8N_{max}}{8}-n}^* & \frac{7N_{max}}{8} < n < \frac{8N_{max}}{8}, \end{cases} \quad (5.2)$$

where C^* denotes the complex conjugate of C .

5.4.1 Twiddle ROM

Twiddle ROM is a read-only memory that stores the real and imaginary parts of the twiddle factors in the first octant. The data should be stored in the fixed-point format. In VHDL

it is possible to generate the exact values and store them in an array. The component has a configurable FFT length, stage index, and the number of fractional bits to use for the fixed-point representation of the twiddle factors.

The memory that is the most suitable for storing the twiddle factors depends on the data width and the number of twiddle factors to store, similar to the FIFO memory type selection described previously.

5.4.2 Twiddle Address Generator

Twiddle address generator is a component that generates a sequence of addresses that contain the appropriate twiddle factor. This is required because the stored values of the twiddle factors are not linearly accessed.

The addresses are in the range of 0 up to $a_{max} = \lceil \frac{N}{8} \rceil + 1$. For naturally-ordered input, the address generator outputs a sequence of 0 to a_{max} and then a sequence of decreasing addresses from $a_{max} - 1$ down to 1, after which the sequence repeats.

Bit-reversed input order requires a slightly more complex addressing scheme, but it can be simplified down to a bit-reverse order counter.

5.4.3 Twiddle Generator

Twiddle generator is a component that takes the output of the twiddle ROM, and makes the necessary operations to transform the twiddle factor that is in the first octant to the required.

The operations that need to be done are swapping the real and imaginary parts and/or changing their sign. The sequence of operations that need to be done is generated from the counter that counts the index of the twiddle factor.

5.4.4 Complex Multiplier

A complex multiplication of two complex numbers can be represented by the following formula:

$$(x + jy) \cdot (C + jS) = (xC - yS) + 1j \cdot (xS + yC) \quad (5.3)$$

This requires four real/fixed-point multiplications, and two additions. Data should be properly pipelined to align the samples that represent the same point in time. The two additions may be performed in the same DSP slices that do a multiplication if the DSP architecture allows for that. For instance, the DSP cores found in Xilinx FPGA devices have an optional adder after the multiplier unit. This way, the two DSP units are cascaded in such a way that the first DSP does an ordinary multiplication (for instance $a \cdot x$), while the second does the multiplication ($b \cdot y$) and an addition of the two results of the multiplication.

A diagram of a DSP core found in Xilinx FPGA devices is given in figure 5.4, and the core components, save for the data widths, have stayed the same for several generations (DSP48E1 [26], DSP48E2 [27] and DSP58 [28]).

An overview of the features implemented in DSP units in different FPGA architectures from Xilinx is given in table 5.2.

The diagram of the 4 DSP cores required for the multiplication is shown in the diagram 5.5, shown with the enabled internal registers used to pipeline the data appropriately. The PCIN and PCOUT are dedicated ports for cascading the output of a DSP slice to the input of the slice above it. This path is also implemented using dedicated routing resources which are the shortest possible for cascading the two slices.

It should be noted that the DSP slices are cascaded using dedicated routing resources, therefore increasing the maximum operating frequency.

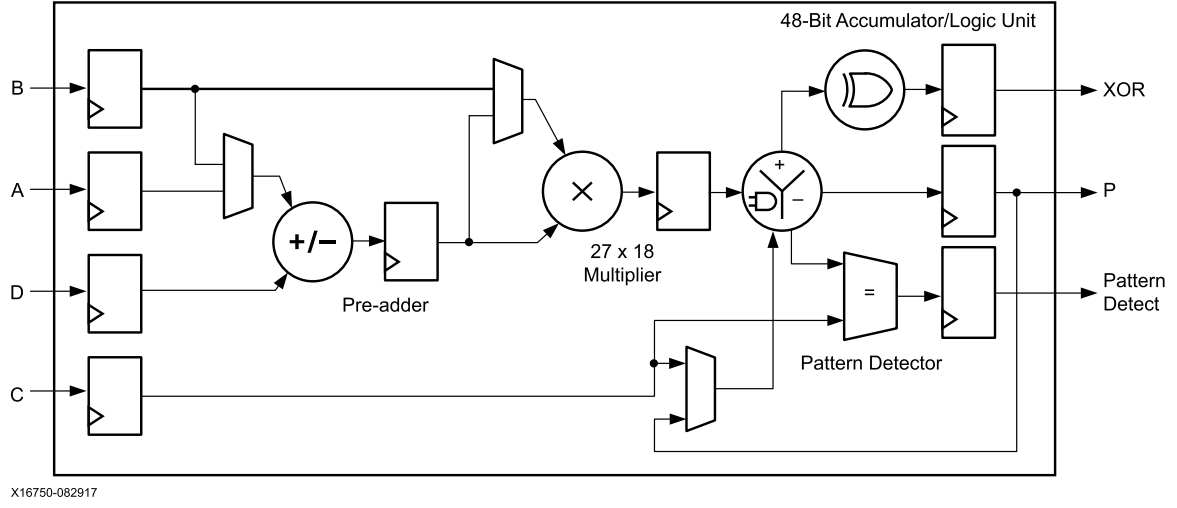


Figure 5.4: A simplified diagram of a DSP48E2 slice [27]

Table 5.2: Features of different DSP generations

Architecture	Hardware multiplier	B input width	Pre-adder output width	Post-adder operands
DSP48	18x18	18	N/A	3
DSP48A1	18x18	18	18	3
DSP48E1	25x18	18	25	3
DSP48E2	27x18	18	27	4
DSP58	27x24	24	27	4

The Vivado synthesis should automatically infer the pipeline registers if the VHDL description matches the shown architecture of the complex multiplier, but care should be taken because there are requirements on clock-enable and reset inputs of the pipeline registers. The exact layout can be verified by opening the synthesized design and reading the `AREG`, `BREG`, `MREG` and `PREG` properties of the DSP units.

If the DSP slice utilization is critical, the number of multipliers can be reduced to three by rearranging the formula 5.3, but would require three more additions.

$$\begin{aligned}
 (x + jy) \cdot (C + jS) &= (xC - yS) + 1j \cdot (xS + yC) \\
 &= (xC + xS - xS - yS) + 1j \cdot (xS + xC - xC + yC) \\
 &= ((C + S)x - (x + y)S) + 1j \cdot ((C + S)x + (y - x)C)
 \end{aligned} \tag{5.4}$$

However, with the appropriate hardware support for a pre-adder on the respective FPGA platform, these additions can be integrated into the same DSP block that does the multiplication. For example, FPGA devices from Xilinx support this since the release of the DSP48A1 DSP core [29] in the Spartan-6 series.

Although the three new additions take no additional resources in that case, data must be properly aligned, and registers external to the DSP cores are necessary. Additionally, the DSP slices cannot be cascaded in a way that was possible for the four-multiplier variant, and therefore additional routing resources must be utilized. This somewhat lowers the maximum possible operating frequency. The diagram of the complex multiplier is shown in figure 5.6.

When it comes to fixed point widths, it is considered that both the x and y data sample real and imaginary components have b_D bits, f_D of which are fractional; while C and S twiddle factor components have b_T bits, f_T of which are fractional.

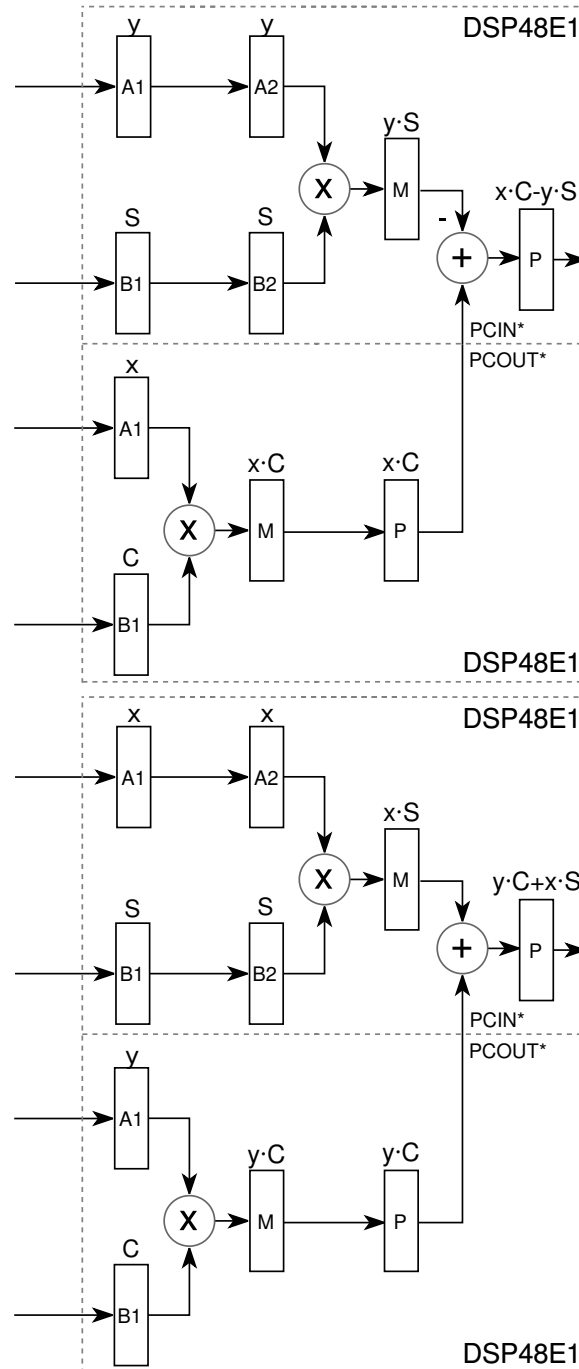


Figure 5.5: A complex multiplier composed of four DSP48 slices

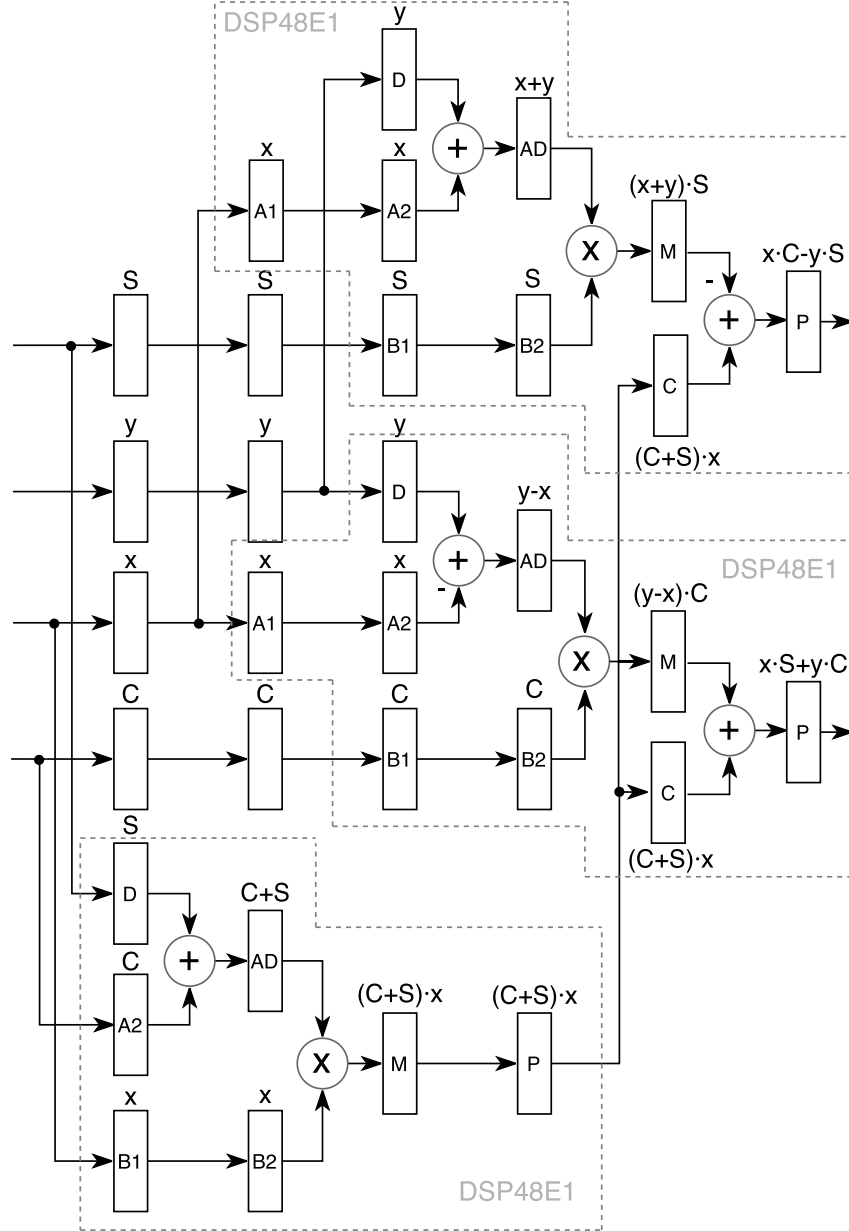


Figure 5.6: A complex multiplier composed of three DSP48 slices [30]

When a four-multiplier implementation is considered, all of the four-multiplier outputs are $b_D + b_T$ bits wide, and have $f_D + f_T$ fractional bits. Cascading the output of one DSP slice to the input of another is allowed since the binary point is in the same position. Input widths of the hardware multiplier determine the maximum supported data and twiddle width.

Starting from the DSP48E2, the post-adder in the DSP slice does a four-operand addition, allowing an additional operand for addition to be introduced. Through the creative use of the C input and OPMODE, one extra bit can be added to one of the operands, making the DSP slice do a multiplication that is larger than the hardware multiplier itself. This relies on the dynamic change of DSP configuration available through the OPMODE input.

However, if the three-multiplier implementation is used, different width limitations exist. First of all, the pre-adder inputs were fixed to A and D DSP inputs for the DSP architectures before DSP48E2. As the B input is wired to S and x, neither of those (the twiddle factor and data sample) can be wider than the B input. For the latter architectures, the sample widths are limited by the width of the pre-adder.

So far, hardware support for rounding has not been discussed. Depending on the design

requirements, different hardware resources can be utilized to implement various types of rounding. The output of the DSP is a full width multiplication output, and there are several practical options for rounding the output down to the required width.

The first option is truncation - propagating a number of most significant bits, and discarding the rest. Discarding a number of most significant bits is also allowed if it can be guaranteed that an overflow will never happen. The overflow analysis should first be performed using a software reference model. Truncation introduces a negative bias to the data samples, causing a lower signal to quantization noise ratio (SQNR). The only advantage of truncation is that it requires no resources to implement, and thus is available on all devices.

The second option is "round half up" rounding. The idea is that adding a constant to the output and then doing a truncation creates an offset that will lower the impact of the negative bias introduced by truncation. This requires adding a constant to the output of the multiplier. The constant's value should be equal to half of the value that the LSB represents after the rounding.

The required constant can be added to the product in the first DSP slice using the **C** input if it is unused. If it is already used for implementing a multiplication that is wider than the width of the hardware multiplier itself, the constant can be programmed into the **RND** register inside the DSP core.

For example, if the output of the multiplier is 48 bits and a 28-bit output is required, 20 fractional bits must be discarded. Simply truncating the bits can lead to an error of up to $2^{20} - 1$ LSBs. If, however, 2^{19} is added to the multiplier output before truncating the bits, the maximum quantization error is reduced to 2^{19} LSBs.

The negative values in two's complement also get rounded up (towards positive infinity) using this method. Such an asymmetric behaviour is sometimes unacceptable, and some form of symmetric rounding is preferred.

Rounding away from zero is the third option that attempts to perform rounding that is symmetric. The difference from the previous rounding mode is only at the halfway point. For instance, the "round half up" method would round the value of -2.5 to -2 , while this method rounds the same number to -3 .

Xilinx DSP48E2 DSP cores have hardware support that simplifies an implementation of different styles of rounding, or preventing overflow/underflow by saturating the output to the largest/smallest number. This is done using a component called "Pattern detector". This component is used to detect a programmable pattern that can occur on the output. Using the pattern detector to detect a condition when the output value is exactly at the midpoint between the two allowed values, logic can be used to modify the output value to implement either convergent rounding or saturation.

For instance, implementing "round half to even" rounding mode using pattern detection in a four-slice complex multiplier is possible by detecting the value of 0 to the right of the binary point. The pattern detector then asserts the **PATTERNDETECT** signal, which can be used to clear the least-significant bit of the output, making it even. One LUT6 is sufficient to implement this logic for both real and imaginary parts of the output. Flip-flop registers are introduced after the look-up table to decrease the "out to clock" timing at the output of the complex multiplier, thereby increasing the maximum usable clock frequency.

As the performance was critical, only the four-slice complex multiplier was implemented.

Chapter 6

Results and Comparison

6.1 Hardware Utilization

In table 6.1 is a comparison of the resource utilization of the implemented SDF architecture IP core and the Xilinx FFT IP core with an equivalent configuration. The implemented configuration is a 1024-point FFT, with 16-bit data and 18-bit twiddle factors. All of the reports are without the data-reordering circuit, and the Xilinx IP core was setup so that the output is in bit-reverse order. It can be seen that the IP core available freely from Xilinx achieves a slightly lower resource utilization. However, the implemented IP core has a number of features that the Xilinx IP core does not support, making it superior in such scenarios.

Table 6.1: Resource utilization of single-path architectures

	Radix-2 ² SDF	Xilinx IP core
LUT	2978	2458
Flip-flop	3971	3943
DSP48E2	16	16
BRAM	2.5	2

Even though the details of implementation were not provided, it can be assumed that the Xilinx FFT IP core also implements the radix-2² SDF algorithm based on the similar utilization.

Multi-path structures, and thus processing multiple samples in parallel, is not provided with the Xilinx IP core. The resource utilization of the implemented MDF architecture configurations is listed in 6.2. The implemented configuration is a 2048-point FFT as a 2-path MDF architecture. The data and twiddle widths are 16 and 18 bits, respectively.

Table 6.2: Resource utilization of the implemented radix-2² MDF architecture FFT

	Natural input order	Bit-reverse input order
LUT	6271	6322
Flip-flop	8372	8466
DSP	40	40
BRAM	7	6

Additionally, the Xilinx IP core does not support processing multiple separate streams in parallel. This refers to calculating FFT of multiple data streams, such that every clock cycle arrives a single sample from several streams; as opposed to the multi-path architectures where several samples from the same data stream every clock cycle arrive. Processing multiple streams in parallel allows the the control logic and twiddle memories to be reused, lowering the per-core

utilization. Utilization of the SDF architecture with 8 parallel streams is listed in table 6.3, and the resource usage is compared to that of 8 independent cores. It can be seen that significant resources can be saved if multiple data streams are processed in parallel.

Table 6.3: Resource utilization of single-path architectures with 8 parallel data streams

	Radix-2 ² SDF	Utilization decrease
LUT	17670	26.1 %
Flip-flop	26653	16.3 %
DSP	128	0 %
BRAM	13	35 %

Another configuration which is not supported by Xilinx IP core is the bit-reverse input data order. This is especially useful when data is processed in the frequency domain and is then converted back to the time domain using an IFFT. If the processing algorithm can be computed in bit-reverse data order, the IFFT can then be used to change the data order from bit-reverse back to natural without the need for data-reorder circuits on both the FFT and the IFFT. Bit-reverse input data order is supported for both single-path and multi-path architectures.

Table 6.4 shows the resource utilization of an FFT that only differs in data order from the FFT shown in 6.1.

Table 6.4: Resource utilization of the single-path architecture FFT with bit-reverse input order

	Radix-2 ² SDF
LUT	2951
Flip-flop	3993
DSP48E2	16
BRAM	2.5

6.2 SQNR Performance

In order to determine the number of bits in the fixed point representation, the fixed point model is compared to the floating point FFT implementation available in the **numpy** library [31]. This isolates only the quantization noise so that signal to quantization noise ratio (SQNR) is calculated. Additionally, the twiddle factors are represented with a sufficiently large number of bits so that only the quantization of data samples causes the error. A comparison of the SQNR at the output of a 512-point FFT for different bit widths from 14 to 20 is given in figure 6.1. A linear relationship can be observed between the number of data bits and the resulting SQNR. For reference, SQNR performance of truncation rounding method is also shown. It performs significantly worse when compared to the cases when rounding is performed.

Results for the SDF architecture software model are shown, but the results are identical for the MDF architecture software model as only the order of operations has changed, while the widths and scaling remain the same.

A comparison of the SQNR performance of FFTs of various sizes is given in figure 6.2. It can be seen that doubling the FFT size decreases the SQNR by about 3dB. This is in line with the results for Strategy 2 from [12].

Finally, the effect of different rounding methods on SQNR performance is presented in figure 6.2. Here a slight advantage of convergent rounding can be observed when compared to the conventional "round-half-up" rounding. However, the conventional rounding is usually easier to implement and takes less resources.

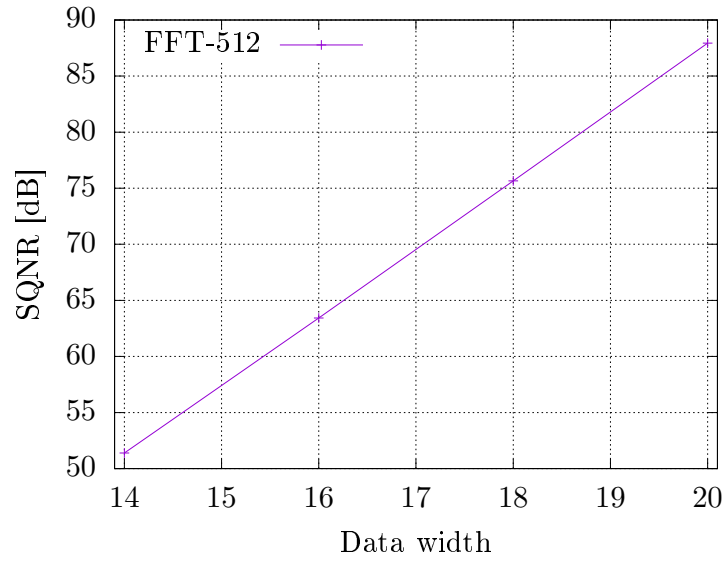


Figure 6.1: SQNR performance of a 512-point FFT for various data widths

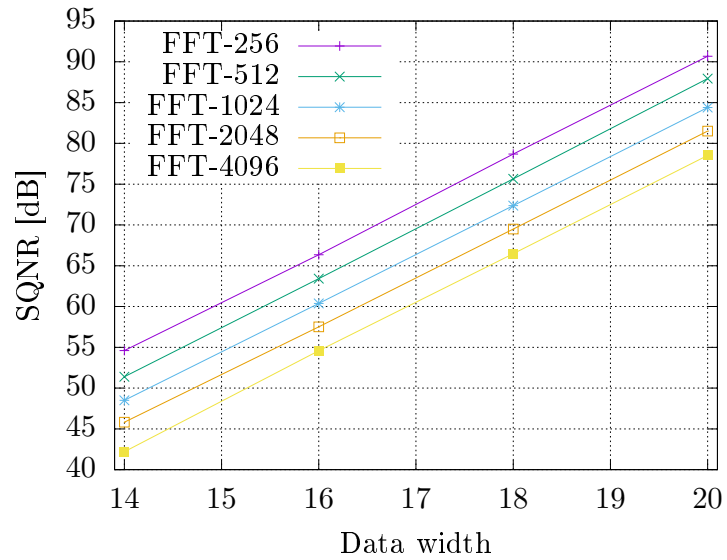


Figure 6.2: SQNR performance of various FFT sizes

So far, the number of fractional bits has been fixed to 24, but the twiddle bit-width also has an effect on SQNR performance, as well as resource utilization. A comparison of different twiddle and input width combinations is shown in figure 6.4 for a 1024-point FFT with convergent rounding. It can be observed that increasing the twiddle width improves the SQNR performance until a certain point, after which it remains fairly constant. Also note that the threshold heavily depends on the width of the input data.

6.3 Testing and Verification

All of the simulations were done using Xilinx Vivado 2021.1, by generating reference files for the input and output using the software reference model, and then comparing the results of the simulation. The bit-exactness of the VHDL to the software reference model was checked for various FFT configurations and test data vectors.

The AXI4-Stream compliance was also tested by randomly generating the `valid` signal in the input bus, and the `ready` signal in the output bus.

The verification of the hardware was performed using `pynq` [32] environment on a ZCU111 development board. This board is based on a Zynq Ultrascale+ chip from Xilinx. The chip is divided into two units - the programmable logic and the processing system. The programmable logic (PL) is the part of the chip that gets reconfigured according to the generated bitstream, while the processing system (PS) consists of several processor cores. The processing system is typically configured to run an operating system, and use the programmable logic to implement the functions that would be impossible (or impractical) to implement in software. An interface between the FPGA logic and the processing system is required, such as a DMA that can access a part of system memory reserved for such purpose. An example design of a FFT connected to the Zynq processing system through a DMA via an AXI4-Stream interface is given in figure 6.5, with the FFT block highlighted.

The `pynq` environment is a GNU/Linux distribution derived from Ubuntu that specifically targets Zynq line. As such, it comes with a number of Python libraries and drivers that make it possible to program and control the programmable logic from Python and Jupyter notebooks. This significantly simplifies the testing and verification as the tests can be written using a high level programming language such as Python.

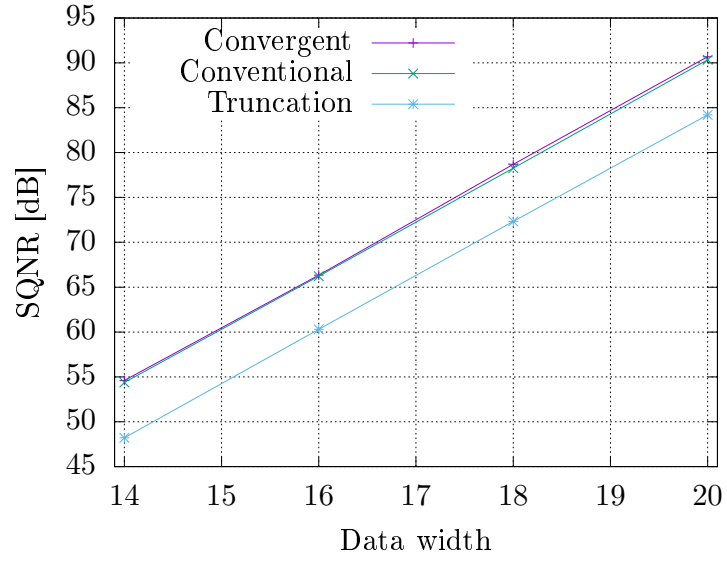


Figure 6.3: SQNR performance of an FFT for different rounding methods

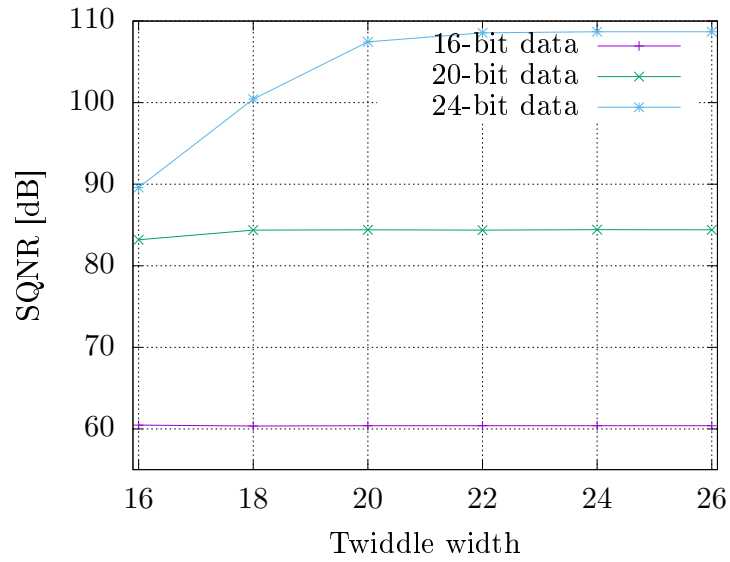


Figure 6.4: SQNR performance of an FFT with respect to twiddle factor width

Chapter 7

Conclusion

This thesis focused on implementing single- and multi-path FFT architectures for processing a high-throughput stream of data, capable enough to be used in a modern telecommunications system whose throughput requirements grow constantly. The thesis also gives a detailed overview of the various hardware architectures with the goal of selecting the architecture to implement. The aim was to show how implementing a custom solution can bring numerous benefits over the freely available solutions, one of which is the additional flexibility in system design phase, especially regarding the data order and parallel processing.

The design targeted contemporary FPGA devices from Xilinx, specifically Zynq Ultrascale+ devices. The design was successfully implemented using Xilinx Vivado 2021.1 and tested on a ZCU111 development board based on the Zynq Ultrascale+ RFSoc ZCU28DR device. The design was implemented at a frequency of 400 MHz, and achieved a throughput of 8 GSPS by processing multiple data streams in parallel.

Along with the hardware implementation, a bit-exact software reference model was developed with the goal of testing the SQNR performance of the design, as well as fine-tuning the parameters until a desired performance target is reached. The procedure of fine-tuning the parameters is thoroughly examined, together with the impact that each parameter has on SQNR performance and resource utilization. The developed software model can easily be integrated into a system-level simulation environment, such as a telecommunications system consisting of multiple components.

The implemented design could possibly further be improved by also implementing saturation logic which could potentially allow for a smaller data width, sacrificing minimal SQNR performance if an overflow occurs rarely. Additionally, there is room for improvement regarding the flexibility of the MDF architecture, as only 2-parallel MDF architecture was implemented. The resource utilization of the design could be improved upon by carefully examining the utilization report, simplifying the critical sections and removing some of the registers which were conservatively placed to help with the timing closure of the design. Also, fine-tuning the type of memory used in each stage could help decrease the power requirements, as well as improve timing of the design.

Bibliography

- [1] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” *Mathematics of computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [2] M. V. Popović, *Digitalna obrada signala*. Akademska misao, 2003.
- [3] R. Lyons. Four ways to compute an inverse FFT using the forward FFT algorithm. (accessed: 01.09.2023). [Online]. Available: <https://www.dsprelated.com/showarticle/800.php>
- [4] E. Dubois and A. Venetsanopoulos, “A new algorithm for the radix-3 FFT,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 3, pp. 222–225, 1978.
- [5] M. Corinthios, K. Smith, and J. Yen, “A parallel radix-4 fast Fourier transform computer,” *IEEE Transactions on Computers*, vol. C-24, no. 1, pp. 80–92, 1975.
- [6] D. Kolba and T. Parks, “A prime factor fft algorithm using high-speed convolution,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 25, no. 4, pp. 281–294, 1977.
- [7] P. Duhamel and H. Hollmann, “Implementation of ”split-radix” FFT algorithms for complex, real, and real symmetric data,” in *ICASSP ’85. IEEE International Conference on Acoustics, Speech, and Signal Processing*, vol. 10, 1985, pp. 784–787.
- [8] R. Singleton, “An algorithm for computing the mixed radix fast Fourier transform,” *IEEE Transactions on Audio and Electroacoustics*, vol. 17, no. 2, pp. 93–103, 1969.
- [9] M. Garrido, “A survey on pipelined FFT hardware architectures,” *Journal of Signal Processing Systems*, vol. 94, no. 11, pp. 1345–1364, 2022.
- [10] S. He and M. Torkelson, “A new approach to pipeline FFT processor,” in *Proceedings of International Conference on Parallel Processing*. IEEE, 1996, pp. 766–770.
- [11] A. V. Oppenheim and R. W. Schaffer, *Digital Signal Processing*. Prentice-Hall International, Inc., 1975.
- [12] P. Gupta, “Accurate performance analysis of a fixed point FFT,” in *2016 Twenty Second National Conference on Communication (NCC)*. IEEE, 2016, pp. 1–6.
- [13] Subclassing ndarray. NumPy. (accessed: 01.09.2023). [Online]. Available: <https://numpy.org/doc/stable/user/basics subclassing.html>
- [14] H. Groginsky and G. Works, “A pipeline fast Fourier transform,” *IEEE Transactions on Computers*, vol. C-19, no. 11, pp. 1015–1019, 1970.
- [15] C. Ingemarsson and O. Gustafsson, “SFF - the single-stream FPGA-optimized feedforward FFT hardware architecture,” *Journal of Signal Processing Systems*, vol. 90, no. 11, pp. 1583–1592, 2018.

- [16] S.-G. Chen, S.-J. Huang, M. Garrido, and S.-J. Jou, “Continuous-flow parallel bit-reversal circuit for MDF and MDC FFT architectures,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 61, no. 10, pp. 2869–2877, 2014.
- [17] *Vivado Design Suite User Guide: Using the Vivado IDE*, Xilinx, Inc., 7 2021. [Online]. Available: <https://docs.xilinx.com/r/2021.1-English/ug893-vivado-ide>
- [18] *AMBA 4 AXI4-Stream Protocol*, ARM, 2010. [Online]. Available: <https://developer.arm.com/documentation/ih0051/a>
- [19] *UltraScale Architecture Memory Resources User Guide*, Xilinx, Inc., 9 2021. [Online]. Available: <https://docs.xilinx.com/v/u/en-US/ug573-ultrascale-memory-resources>
- [20] *UltraScale Architecture Configurable Logic Block User Guide*, Xilinx, Inc., 2 2017. [Online]. Available: <https://docs.xilinx.com/v/u/en-US/ug574-ultrascale-clb>
- [21] *AXI4-Stream Infrastructure IP Suite*, Xilinx, Inc., 5 2023. [Online]. Available: <https://docs.xilinx.com/v/u/en-US/ug574-ultrascale-clb>
- [22] *FIFO Intel FPGA IP User Guide*, Intel Corporation, 4 2023. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/programmable/683522.html>
- [23] M. Garrido, F. Qureshi, J. Takala, and O. Gustafsson, *Hardware architectures for the fast Fourier transform*. Cham: Springer International Publishing, 2019, pp. 613–647. [Online]. Available: https://doi.org/10.1007/978-3-319-91734-4_17
- [24] M. Hasan and T. Arslan, “FFT coefficient memory reduction technique for OFDM applications,” in *2002 IEEE International Conference on Acoustics, Speech, and Signal Processing*, vol. 1, 2002, pp. I–1085–I–1088.
- [25] T. Patyk, F. Qureshi, and J. Takala, “Hardware-efficient twiddle factor generator for mixed radix-2/3/4/5 FFTs,” in *2016 IEEE International Workshop on Signal Processing Systems (SiPS)*, 2016, pp. 201–206.
- [26] *7 Series DSP48E1 Slice User Guide*, Xilinx, Inc., 3 2018. [Online]. Available: https://docs.xilinx.com/v/u/en-US/ug479_7Series_DSP48E1
- [27] *UltraScale Architecture DSP Slice User Guide*, Xilinx, Inc., 8 2021. [Online]. Available: <https://docs.xilinx.com/v/u/en-US/ug579-ultrascale-dsp>
- [28] *Versal ACAP DSP Engine Architecture Manual*, Xilinx, Inc., 9 2022. [Online]. Available: <https://docs.xilinx.com/r/en-US/am004-versal-dsp-engine>
- [29] *Spartan-6 FPGA DSP48A1 Slice User Guide*, Xilinx, Inc., 5 2014. [Online]. Available: <https://docs.xilinx.com/v/u/en-US/ug389>
- [30] M. Garrido and P. Malagón, “The constant multiplier FFT,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, no. 1, pp. 322–335, 2021.
- [31] Discrete Fourier transform. NumPy. (accessed: 01.09.2023). [Online]. Available: <https://numpy.org/doc/stable/reference/routines.fft.html>
- [32] Pynq - Python productivity for Zynq. AMD. (accessed: 01.09.2023). [Online]. Available: <http://www.pynq.io/>

List of Abbreviations

FPGA	Field-Programmable Gate Array
FFT	Fast Fourier Transform
IP	Intellectual Property
ASIC	Application Specific Integrated Circuit
DFT	Discrete Fourier Transform
IDFT	Inverse Discrete Fourier Transform
IFFT	Inverse Fast Fourier Transform
DIF	Decimation in Frequency
DIT	Decimation in Time
MSB	Most significant bit
LSB	Least significant bit
SDF	Single-path delay feedback
MDF	Multi-path delay feedback
SFF	Single-path Feed-forward
RTL	Register-Transfer Level
RAM	Random-Access Memory
ROM	Read-only memory
DSP	Digital Signal Processing
FIFO	First-In First-Out
SQNR	Signal-to-Quantization-Noise Ratio
CORDIC	Coordinate Rotation Digital Computer
PS	Processing System
PL	Programmable Logic
DMA	Direct Memory Access
ILA	Integrated Logic Analyzer

List of Figures

2.1	Signal flow graph of an FFT operation split into 2 smaller FFTs [2]	7
2.2	Butterfly of a radix-2 DIF FFT [2]	8
2.3	Signal flow graph of a DIF FFT [2]	8
2.4	Signal flow graph of a DIF FFT with the input samples in bit-reverse order [2]	9
2.5	Signal flow graph of a DIT algorithm [2]	10
2.6	Butterfly of a radix-2 DIT FFT [2]	10
2.7	IFFT using the forward FFT by conjugating the data [3]	10
2.8	IFFT using the forward FFT by swapping the real and imaginary parts [3]	11
2.9	Radix-2 ² algorithm derived from radix-2	12
2.10	Merging two subsequent multiplications in radix-2 ² butterflies	13
3.1	Software model architecture	17
4.1	Radix-2 SDF FFT architecture [9]	20
4.2	Signal flow in an SDF stage [9]	20
4.3	A memory inefficient FFT stage architecture	21
4.4	SDF stage architecture [9]	21
4.5	SDF FFT with a bit-reverse input order [9]	22
4.6	Data flow of an SFF stage	22
4.7	SFF stage architecture	22
4.9	MDF FFT architecture [9]	24
4.10	Data flow graph used to derive MDF architecture with bit-reverse input order	25
4.11	MDF FFT architecture with bit-reverse input order	25
4.12	Binary representation of the sample index used for parallel bit-reversal [16]	27
4.13	Parallel bit-reversal circuit [16]	28
5.1	An FFT composed of 4 stages	30
5.2	Architecture of the block RAM FIFO in Ultrascale+ architecture [19]	31
5.3	Twiddle factors on a unit circle [23]	32
5.4	A simplified diagram of a DSP48E2 slice [27]	34
5.5	A complex multiplier composed of four DSP48 slices	35
5.6	A complex multiplier composed of three DSP48 slices [30]	36
6.1	SQNR performance of a 512-point FFT for various data widths	40
6.2	SQNR performance of various FFT sizes	40
6.3	SQNR performance of an FFT for different rounding methods	42
6.4	SQNR performance of an FFT with respect to twiddle factor width	42
6.5	Block design for testing the FFT	43

List of Tables

5.1	An overview of the signals in the AXI4-Stream protocol	29
5.2	Features of different DSP generations	34
6.1	Resource utilization of single-path architectures	38
6.2	Resource utilization of the implemented radix-2 ² MDF architecture FFT	38
6.3	Resource utilization of single-path architectures with 8 parallel data streams . .	39
6.4	Resource utilization of the single-path architecture FFT with bit-reverse input order	39